Lattice-Based Post-Quantum Cryptography

BSc Research Internship Report

Alexandre Autran
École Normale Supérieure de Rennes

Supervised by Aurore Guillevic and Pierre Alain Fouque
from the CAPSULE team at Inria and IRISA
Applied Cryptography and Implementation Security

Inria – University of Rennes
National Institute for Research in Digital Science
and Technology

June–August 2025

Acknowledgements

I would like to thank Aurore Guillevic and Pierre-Alain Fouque for agreeing to supervise this internship, as well as for all their explanations, which enabled me to discover the field of post-quantum cryptography. Finally, I thank the Inria Centre at the University of Rennes for hosting me during June, July, and August.

Internship Context (Overview)

Inria (the French National Institute for Research in Digital Science and Technology) is a French public research institution dedicated to computer science, automation, and applied mathematics. It collaborates with a range of academic and industrial partners and is organised into regional centres, including the Rennes centre. It operates under multi-year performance agreements with the French government.

IRISA (the Research Institute in Computer Science and Random Systems) is a joint research unit (UMR) bringing together several supervisory institutions (CNRS, Inria, ENS, the University of Rennes, etc.). Its activities cover a broad range of computer-science topics, organised into seven major scientific areas.

The CAPSULE team (Applied Cryptography and Implementation Security) is a recently established joint Inria–IRISA project team, created in January 2023. Its research focuses on cryptography, particularly resistance to quantum attacks, cryptanalysis, and implementation security in practical settings involving hardware, software, and protocols.

Introduction

Post-quantum cryptography encompasses cryptographic systems designed to withstand attacks by quantum computers while remaining executable on classical computers. It differs from both:

- *Classical cryptography*, which relies on security assumptions that are vulnerable to quantum algorithms, notably Shor's algorithm, which breaks RSA, DSA, and ECDSA by solving integer factorisation and discrete logarithms in polynomial time, and Grover's algorithm, which accelerates exhaustive searches.
- *Quantum cryptography*, which uses the laws of quantum physics (e.g. quantum key distribution–QKD) to secure communications and requires quantum hardware.

Post-quantum cryptography therefore remains entirely classical in its operation, requiring no quantum devices, but relies on problems believed to remain difficult even for a quantum computer, such as lattice problems (SIS/LWE), error-correcting codes, elliptic-curve isogenies, and multivariate problems.

As part of the preparation of new "post-quantum" cryptographic algorithms (that is, algorithms capable of withstanding attacks by quantum computers), the NIST (National Institute of Standards and Technology) launched an international competition in 2016 covering signature schemes and key-encapsulation mechanisms. The first standards were published in August 2024: ML-DSA for signatures, based on Dilithium, and ML-KEM for encryption, based on Kyber. The FIPS-206 standard, which will be based on the Falcon proposal, is expected in 2025–2026. The security of these schemes relies on hard problems defined over lattices, which this report sets out to study.

Contents

References

[1] Erdem Alkim, Joppe W. Bos, Léo Ducas, Lewis Glabush, Patrick Longa, Ilya Mironov, Michael Naehrig, Valeria Nikolaenko, Chris Peikert, Ananth Raghunathan, and Douglas Stebila. FrodoKEM. Technical report, NIST submission, 2024. https://frodokem.org/.

[2] Erdem Alkim, Léo Ducas, Thomas Pöppelmann, and Peter Schwabe. Post-quantum key exchange – A new hope. In Thorsten Holz and Stefan Savage, editors, *25th USENIX Security Symposium, USENIX Security 16, Austin, TX, USA, August 10-12, 2016*, pages 327–343. USENIX Association, 2016. ePrint:2015/1092.

[3] Shi Bai, Léo Ducas, Eike Kiltz, Tancrède Lepoint, Vadim Lyubashevsky, Peter Schwabe, Gregor Seiler, and Damien Stehlé. CRYSTALS-Dilithium: Algorithm Specifications and Supporting Documentation (Version 3.1). `https://pq-crystals.org/dilithium`, 2023. https://pq-crystals.org/dilithium.

[4] Léo Ducas, Eike Kiltz, Tancrède Lepoint, Vadim Lyubashevsky, Peter Schwabe, Gregor Seiler, and Damien Stehlé. CRYSTALS-Dilithium: A Lattice-Based Digital Signature Scheme. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2018(1):238–268, 2018.

[5] Alfred Menezes. *Cryptography 101*. University of Waterloo, ON, Canada, 2025. https://cryptography101.ca/.

[6] National Institute of Standards and Technology NIST. Sha-3 standard: Permutation-based hash and extendable-output functions. Standard FIPS PUB 202, US department of commerce, August 2015. http://dx.doi.org/10.6028/NIST.FIPS.202.

[7] National Institute of Standards and Technology NIST. Module-lattice-based digital signature standard. Standard FIPS 204, US department of commerce, August 13 2024. https://csrc.nist.gov/pubs/fips/204/final.

[8] National Institute of Standards and Technology NIST. Module-lattice-based key-encapsulation mechanism standard. Standard FIPS 203, US department of commerce, August 13 2024. https://csrc.nist.gov/pubs/fips/203/final.

[9] Ian N. Stewart and David O. Tall. *Algebraic Number Theory and Fermat's Last Theorem*. Chapman and Hall/CRC, 4th edition, October 2015. Textbook - 322 Pages - 21 B/W Illustrations.

1 Euclidean Lattices

1.1 Definitions and Basic Properties

In this section, we introduce the foundations of lattice theory and two algorithmic problems arising from it: SVP (Shortest Vector Problem) and CVP (Closest Vector Problem). In addition, to allow approximate solutions and characterise their difficulty, we also define the problems SVP_γ and GapSVP_γ.

To attack these problems, we use the Lenstra–Lenstra–Lovász (LLL) algorithm, which transforms an arbitrary basis in polynomial time into a "weakly reduced" basis whose vectors are relatively short and nearly orthogonal. LLL therefore provides an efficient means of generating approximate solutions to SVP and CVP.

Definition 1.1 (Lattice and lattice basis)
A *Euclidean lattice* $\mathscr{L}$ is the set of all integer linear combinations of m vectors $b_1, b_2, \cdots, b_m \in \mathbf{R}^n$ that are linearly independent over $\mathbf{R}$. The family $B := \{b_1, b_2, \cdots, b_m\}$ is a *basis* of $\mathscr{L}$, and we write

$$\mathscr{L} = \mathscr{L}(B).$$

The number n is the *dimension* of $\mathscr{L}$, and the number m is its *rank*.

Remarks.

† In what follows, we shall say "a lattice in $\mathbf{R}^n$" (to specify the dimension of the lattice) rather than "Euclidean lattice".

† From now on, we assume that all basis vectors $v_1, \cdots, v_m$ belong to $\mathbf{Z}^n$. In this case, $\mathscr{L}$ is said to be *integral*, and we have

$$\mathscr{L} = \left\{ \sum_{i=1}^{m} x_i b_i \;\middle|\; x_i \in \mathbf{Z} \right\} \subseteq \mathbf{Z}^n.$$

† If we write $B = \begin{pmatrix} b_1 & b_2 & \cdots & b_m \end{pmatrix} \in \mathscr{M}_{n,m}(\mathbf{Z})$, then $\mathscr{L} = \{Bx \mid x \in \mathbf{Z}^m\}$.

A lattice may also be defined as a discrete additive subgroup of $\mathbf{R}^n$. The following theorem states that this definition is equivalent:

Theorem 1.2 (Fundamental theorem of lattices)
Let $\mathscr{L} \subseteq \mathbf{R}^n$ be an additive subgroup of $\mathbf{R}^n$. Then $\mathscr{L}$ is a lattice if and only if $\mathscr{L}$ is discrete.

Definition 1.3 (Full-rank lattice)
A lattice $\mathscr{L} \subseteq \mathbf{R}^n$ is said to have *full rank* if its rank is n, that is, if it has a basis consisting of n independent vectors in $\mathbf{R}^n$.

Definition 1.4 (Sublattice)
Let $\mathscr{L}$ and $\mathscr{L}'$ be two lattices in $\mathbf{R}^n$. We say that $\mathscr{L}'$ is a *sublattice* of $\mathscr{L}$ if

$$\mathscr{L}' \subseteq \mathscr{L}.$$

Remarks.

† Unless stated otherwise, all lattices and sublattices considered are assumed to be full-rank and integral.

† If $B = (b_1, \cdots, b_n)$ is a basis of a full-rank lattice $\mathscr{L} \subseteq \mathbf{R}^n$, then B is also a basis of $\mathbf{R}^n$.

Example. Let $n = 2$ and let $B_1 := \{(1,0), (0,1)\}$. Then

$$\mathscr{L}_1 = \mathscr{L}(B_1) = \{B_1 x \mid x \in \mathbf{Z}^2\}, \quad \text{where} \quad B_1 = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}.$$

More precisely, $\mathscr{L}_1 = \mathbf{Z}^2$.

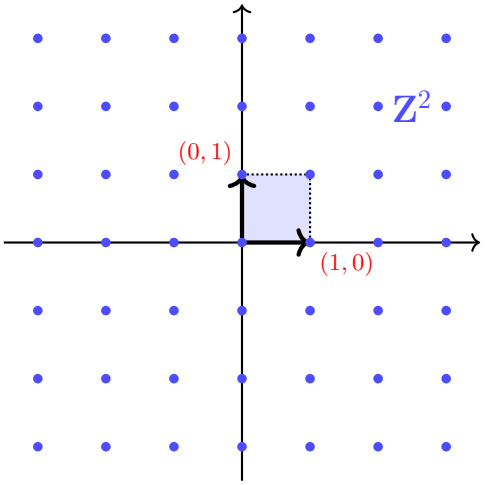


Figure 1: The lattice $\mathbf{Z}^2$, generated by $B_1 = ((1,0),(0,1))$.

Definition 1.5 (Fundamental parallelepiped)

Let $\mathscr{L} = \mathscr{L}(B)$ be a lattice in $\mathbf{R}^n$, where $B = (b_1, b_2, \cdots, b_n)$ is a basis of $\mathscr{L}$. The *fundamental parallelepiped* associated with B is the set of linear combinations with real coefficients in the interval $[0,1]$:

$$\mathscr{P}(B) = \left\{ \sum_{i=1}^{n} a_i b_i \;\middle|\; a_i \in [0,1] \right\}.$$

Remarks.

† Equivalently:

$$\mathscr{P}(B) = \{Bx \mid x \in [0,1]^n\}.$$

† The copies of $\mathscr{P}(B)$ translated by the vectors of $\mathscr{L}(B)$ partition $\mathbf{R}^n$ into non-overlapping regions (parallelepipeds).

† The "vertices" of these parallelepipeds are exactly the points of the lattice $\mathscr{L}(B)$.

Theorem 1.6 (Characterisation of lattice bases)

Let $\mathscr{L} = \mathscr{L}(B_1)$ be a full-rank lattice in $\mathbf{R}^n$, with $B_1 \in \mathscr{M}_n(\mathbf{Z})$. An integer matrix $B_2 \in \mathscr{M}_n(\mathbf{Z})$ is another basis of $\mathscr{L}$ if and only if there exists a unimodular matrix $U \in \mathrm{GL}_n(\mathbf{Z})$ (that is, $\det U = \pm 1$) such that

$$B_1 = B_2 U.$$

Proof.

$\boxed{\Longrightarrow}$ Suppose that B_1 and B_2 are both bases of $\mathscr{L} \subset \mathbf{R}^n$. Since the columns of B_2 belong to $\mathscr{L} = L(B_1)$, there exists an invertible integer matrix $U \in \mathscr{M}_n(\mathbf{Z})$ such that

$$B_2 = B_1 U.$$

Similarly, the columns of B_1 belong to $L(B_2)$, so there exists an invertible matrix $V \in \mathscr{M}_n(\mathbf{Z})$ such that

$$B_1 = B_2 V.$$

Hence $B_1 = B_1(UV)$. Since B_1 is invertible, we obtain $UV = I_n$, and therefore $\det(U)\det(V) = 1$. Thus $\det(U) = \pm 1$ and $\det(V) = \pm 1$, which shows that U is unimodular.

$\boxed{\Longleftarrow}$ Conversely, suppose that there exists a unimodular matrix $U \in \mathscr{M}_n(\mathbf{Z})$ such that

$$B_1 = B_2 U.$$

Since U is invertible in $\mathscr{M}_n(\mathbf{R})$, we have

$$B_2 = B_1 U^{-1},$$

and U^{-1} also has integer entries (because $\det U = \pm 1$). Thus, the columns of B_2 generate the same integer linear combinations as those of B_1, so $L(B_2) = L(B_1)$. Therefore, B_2 is indeed a basis of $\mathscr{L}$. □

Example. Let

$$B_2 = \begin{pmatrix} 2 & 0 \\ 0 & 1 \end{pmatrix} \quad \text{and} \quad B_3 = \begin{pmatrix} -2 & 4 \\ -2 & 3 \end{pmatrix}.$$

Then B_2 and B_3 are bases of the same lattice (see Figure 2), because

$$\begin{pmatrix} 2 & 0 \\ 0 & 1 \end{pmatrix} = \begin{pmatrix} -2 & 4 \\ -2 & 3 \end{pmatrix} \cdot \underbrace{\begin{pmatrix} 3 & -2 \\ 2 & -1 \end{pmatrix}}_{U},$$

where $U \in \mathrm{GL}_2(\mathbf{Z})$ is unimodular ($\det U = 3 \cdot (-1) - (-2) \cdot 2 = -3 + 4 = 1$).

Definition 1.7 (Volume of a lattice)
Let $\mathscr{L} = \mathscr{L}(B)$ be a full-rank lattice in $\mathbf{R}^n$, where $B \in \mathscr{M}_n(\mathbf{Z})$ is a basis of $\mathscr{L}$. The *volume* of $\mathscr{L}$ is

$$\mathrm{vol}(\mathscr{L}) = |\det(B)|.$$

Remarks. The volume of a lattice is the volume of its fundamental parallelepiped.

- † If the lattice has dimension 2, then $\mathrm{vol}(\mathscr{L})$ coincides with the area of its fundamental parallelepiped.
- † Formally, the volume is inversely proportional to the density of lattice points: the larger $\mathrm{vol}(\mathscr{L})$ is, the sparser the lattice.

Proposition 1.8
The volume of a lattice $\mathscr{L} = \mathscr{L}(B)$ does not depend on the choice of basis B.

Proof. Let B_1 and B_2 be two bases of $\mathscr{L}$. By the characterisation of bases, there exists a unimodular matrix $U \in \mathscr{M}_n(\mathbf{Z})$ such that $B_1 = B_2 U$, and $\det U = \pm 1$. Taking determinants gives

$$\det(B_1) = \det(B_2)\det(U) = \pm \det(B_2),$$

hence

$$|\det(B_1)| = |\det(B_2)|.$$

Thus $\mathrm{vol}(\mathscr{L}) = |\det(B)|$ is the same for every basis of $\mathscr{L}$. □

Proposition 1.9 (Volume and sublattices)
If $\mathscr{L}_1 \subseteq \mathscr{L}_2$ are two full-rank sublattices of $\mathbf{R}^n$, then

$$\mathrm{vol}(\mathscr{L}_1) \geqslant \mathrm{vol}(\mathscr{L}_2).$$

Proof. Since $\mathscr{L}_1 \subseteq \mathscr{L}_2$, each vector of the basis B_1 can be written as an integer linear combination of the vectors of B_2. Therefore, there exists a matrix $M \in \mathscr{M}_n(\mathbf{Z})$ such that $B_1 = B_2 M$. Taking determinants gives

$$\det(B_1) = \det(B_2)\det(M).$$

Since $\det(M) \in \mathbf{Z}\backslash\{0\}$, we have $|\det(M)| \geqslant 1$. Hence

$$\mathrm{vol}(\mathscr{L}_1) = |\det(B_1)| = |\det(B_2)||\det(M)| \geqslant |\det(B_2)| = \mathrm{vol}(\mathscr{L}_2),$$

which completes the proof. □

1.2 Lattice Problems

Definition 1.10 (SVP, the shortest vector problem)
Let $\mathscr{L} = \mathscr{L}(B) \subseteq \mathbf{Z}^n$ be a lattice. The *shortest vector problem* (SVP) consists of finding a non-zero vector of

minimum length in $\mathscr{L}$, that is, a vector $v \in \mathscr{L}\backslash\{0\}$ such that

$$\|v\| = \min_{w \in \mathscr{L}\backslash\{0\}} \|w\|.$$

Remarks.

† The shortest vector problem (SVP) is NP-hard.

† The best known classical algorithm for SVP has running time

$$2^{0.292n+o(n)}.$$

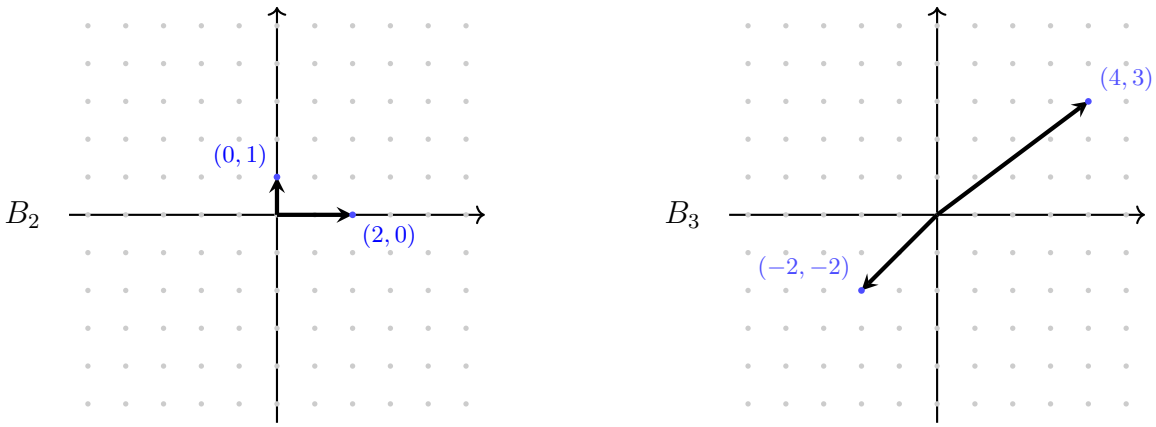
† The best known quantum algorithm for SVP has running time

$$2^{0.265n+o(n)}.$$

Example. Consider the following two SVP instances in dimension 2:

$$\mathscr{L}_2 = \mathscr{L}\big(\{(2,0),(0,1)\}\big) \quad \text{and} \quad \mathscr{L}_3 = \mathscr{L}\big(\{(-2,-2),(4,3)\}\big).$$

The lattices $\mathscr{L}_2$ and $\mathscr{L}_3$ define the same lattice but are described by two different bases. In $\mathscr{L}_2$, the shortest non-zero vector is $(0,1)$ and has norm 1, whereas in $\mathscr{L}_3$, the shortest non-zero vector found is $(-2,-2)$ and has norm $2\sqrt{2}$.
Thus, the practical difficulty of solving SVP on $\mathscr{L}(B)$ depends strongly on the "quality" of the basis B chosen for the lattice. Indeed, the basis $B_2 = \{(2,0),(0,1)\}$ is of better "quality" than the basis $B_3 = \{(-2,-2),(4,3)\}$, because its vectors are shorter and mutually orthogonal.



$\mathscr{L}_2$ generated by $((2,0),(0,1))$ $\mathscr{L}_3$ generated by $((-2,-2),(4,3))$

Figure 2: Two examples of sublattices of $\mathbf{Z}^2$.

Definition 1.11 (Problem SVP_γ (approximation of the shortest vector))

Let $\gamma(n) > 1$ be a real number and let B be a basis of a lattice $\mathscr{L} \subseteq \mathbf{Z}^n$. The problem SVP_γ consists of finding a non-zero vector $v \in \mathscr{L}$ such that

$$\|v\|_2 \leqslant \gamma(n)\lambda_1(\mathscr{L}),$$

where $\lambda_1(\mathscr{L})$ denotes the length of the shortest vector in $\mathscr{L}$.

Remarks.

† The problem SVP_γ is NP-hard.

† When $\gamma = 2^k$, the best known algorithm for SVP_γ runs in time $2^{\widetilde{O}(n/k)}$, where $\widetilde{O}$ hides a factor that is polynomial in the logarithm of n.

† If $\gamma > 2^{(n \log\log n)/\log n}$, then SVP_γ can be solved in time polynomial in the input size using the LLL algorithm (see Definition 1.3.2).

† The fastest known algorithm for solving SVP_γ is the BKZ (Block-Korkine-Zolotarev) algorithm, whose running time is exponential.

Its associated decision problem is the following:

Definition 1.12 (Problem GapSVP$_\gamma$ (decision version))

Let $\gamma(n) > 1$ be a real number and let B be a basis of a lattice $\mathscr{L} \subseteq \mathbf{Z}^n$. The problem GapSVP_γ consists of deciding whether $\lambda_1(\mathscr{L}) \leqslant 1$ or $\lambda_1(\mathscr{L}) \geqslant \gamma(n)$, under the promise that one of these two cases holds.

Definition 1.13 (CVP (Closest Vector Problem))

Let B be a basis of a lattice $\mathscr{L} \subseteq (\mathbf{Z}_q)^n$, and let $x \in (\mathbf{Z}_q)^n$ be a vector. The CVP problem consists of finding a vector $v \in \mathscr{L}$ that minimises $\|v - x\|_2$.

Definition 1.14 (Successive minima of a lattice)

Let $\mathscr{L} \subseteq \mathbf{R}^n$ be a full-rank lattice. For each $i \in \{1, \cdots, n\}$, the i-th *successive minimum* $\lambda_i(\mathscr{L})$ is defined as the smallest real number $r > 0$ such that $\mathscr{L}$ contains i linearly independent vectors of norm at most r.

Remarks.

† It is immediate that $0 < \lambda_1(\mathscr{L}) \leqslant \lambda_2(\mathscr{L}) \leqslant \cdots \leqslant \lambda_n(\mathscr{L})$.

† $\lambda_1(\mathscr{L})$ is exactly the norm of a shortest non-zero vector in $\mathscr{L}$. SVP can therefore be reformulated as follows: given a lattice $\mathscr{L}$, determine a lattice vector of norm $\lambda_1(\mathscr{L})$.

We now state two results:

Proposition 1.15 (Minkowski's theorem)

If $\mathscr{L}$ is a full-rank lattice in $\mathbf{R}^n$, then

$$\lambda_1(\mathscr{L}) \leqslant \sqrt{n}\big(\mathrm{vol}(\mathscr{L})\big)^{1/n}.$$

Proposition 1.16 (Gaussian heuristic)

If $\mathscr{L}$ is a full-rank lattice in $\mathbf{R}^n$, then

$$\lambda_1(\mathscr{L}) \approx \sqrt{\tfrac{n}{2\pi e}}\big(\mathrm{vol}(\mathscr{L})\big)^{1/n}.$$

1.3 LLL and BKZ Algorithms

Definition 1.17 (LLL reduction)

Let $B = (b_1, \cdots, b_n)$ be a basis of a lattice in $\mathbf{R}^n$. Let b_i^* and $\mu_{i,j}$ denote, respectively, the vectors and coefficients obtained from Gram–Schmidt orthogonalisation (see Definition 1.3.2). The basis B is said to be (η, δ)-*LLL-reduced* when $\frac{1}{4} < \delta < 1$ and $\frac{1}{2} \leqslant \eta < \sqrt{\delta}$ and, for every $1 \leqslant j < i \leqslant n$,

$$|\mu_{i,j}| \leqslant \eta \quad \text{(weakly reduced)},$$

and, for every $1 \leqslant i < n$,

$$\delta\|b_i^*\|^2 \leqslant \left\|b_{i+1}^* + \mu_{i+1,i} b_i^*\right\|^2 \quad \text{(Lovász condition)}.$$

Remarks.

† η controls size reduction, that is, the magnitude of the Gram–Schmidt coefficients $\mu_{i,j}$. The smaller η is, the smaller the projections of the vectors onto the preceding subspaces are, and hence the closer the basis vectors are to being orthogonal. In standard LLL, one often takes $\eta = 1/2$.

† δ controls the Lovász condition, which imposes a controlled progression in the norms of the Gram–Schmidt vectors. Typically, δ is chosen close to 1, for example $\delta = 0.99$. The closer δ is to 1, the higher the quality of the resulting basis (with shorter vectors). The smaller δ is, the faster the algorithm, but the longer the vectors it produces.

Definition 1.18 (LLL algorithm)

The Lenstra–Lenstra–Lovász (LLL) algorithm computes, in polynomial time, a relatively short LLL-reduced basis $B^* = (b_1^*, b_2^*, \cdots, b_n^*)$ of a lattice $\mathscr{L} \subseteq \mathbf{Z}^n$ from a basis $B = (b_1, b_2, \cdots, b_n)$.

LLL Algorithm

```
Input: a basis B = (b_1, ..., b_n)
Output: an LLL-reduced basis B* = (b_1*, b_2*, ..., b_n*) obtained from B
LLL(B) :
B* ⟵ WeakReduce(B)                          ▷ Perform weak reduction on B (see the following algorithm)
For i = 1 to n do                           ▷ Perform Gram–Schmidt orthogonalisation
    For j = 1 to i − 1 do
        μ_{i,j} ⟵ ⟨b_i*, b_j*⟩ / ⟨b_j*, b_j*⟩
    End For
    b_i* ⟵ b_i* − Σ_{j<i} μ_{i,j} b_j*
End For
If B* is LLL-reduced then
    Return B*
Else                                        ▷ Swap the vectors of B* to satisfy the inequality
    i ⟵ min{ j ∈ {1, ..., n} | ‖b_{j+1}* + μ_{j+1,j} b_j*‖^2 < δ‖b_j*‖^2 }
    swap(b_i*, b_{i+1}*)
End If
Return B*
```

Weak-Reduction Algorithm (WeakReduce)

```
Input: a basis B = (b_1, ..., b_n)
Output: a weakly reduced basis B* = (b_1*, b_2*, ..., b_n*) obtained from B
(b_1*, ..., b_n*) ⟵ GramSchmidt(B)          ▷ Retrieve the vectors and scalars obtained from Gram–Schmidt
For every pair (i, j) do
    μ_{i,j} ⟵ ⟨b_i, b_j*⟩ / ⟨b_j*, b_j*⟩
End For
If for every pair (i, j), |μ_{i,j}| ⩽ η then
    Return B*
Else                                        ▷ Modify the b_i* so that |μ_{i,j}| ⩽ η
    Let (i, j) be the largest pair in lexicographic order such that |μ_{i,j}| > η
    μ ⟵ the nearest integer to μ_{i,j}
    b_i* ⟵ b_i* − μ b_j
End If
Return B*
```

Remarks.

† The time complexity of the LLL algorithm is $\mathscr{O}\big(d^5 n \log^3(M)\big)$ when LLL takes as input d vectors forming a basis of a lattice of dimension n, each of norm at most M. Here, because we have assumed that the lattices have full rank, the time complexity of the LLL algorithm is $\mathscr{O}(n^6 \log^3(M))$.

† Let $B := \{b_1, \cdots, b_n\}$ be a basis of a lattice in $\mathbf{R}^n$. Let $\{b_1^*, \cdots, b_n^*\}$ denote the LLL-reduced basis obtained from B. Then, for every $1 \leqslant i < n$,

$$\|b_i^*\| \leqslant (\delta - \eta^2)^{1/2} \|b_{i+1}^*\|.$$

This yields the following upper bound for $\|b_1^*\|$:

$$\|b_1^*\| \leqslant (\delta - \eta^2)^{(n-1)/4} \mathrm{vol}(L)^{1/n}.$$

Moreover, the vectors of the basis B approximate $\lambda_i(L)$ within a bounded factor:

$$\|b_i^*\| \leqslant (\delta - \eta^2)^{(n-1)/2} \lambda_i(L).$$

This implies that b_1^* provides a solution to $\mathrm{SVP}_{(\delta-\eta^2)^{(n-1)/2}}$. For $(\eta, \delta) \approx (0.5, 0.999)$, the approximation factor is at most 1.075^{n-1}. In practice, for a random lattice, the average factor is smaller.

Example.

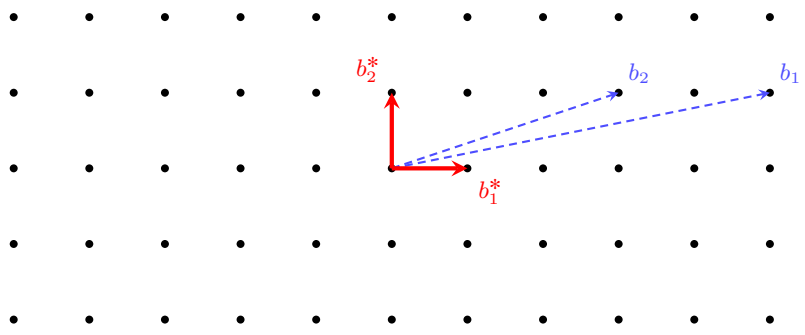


Figure 3: A lattice generated by a *poor* basis (b_1, b_2) (in blue) and by a *good* basis (b_1^*, b_2^*) (in red).

We now study a possible improvement of LLL introduced by Schnorr and Euchner in 1991: the Blockwise-Korkine-Zolotarev (BKZ) algorithm. The principle is as follows: BKZ uses a block parameter B_0 ($2 \leqslant B_0 \leqslant n$). The key idea is to apply stronger reductions locally, block by block, while retaining the global structure of LLL: at each step, one considers a sublattice of dimension B_0 (a block) and applies a strong reduction, using SVP, within that block. The algorithm then moves through the basis by sliding the block window. This process continues until stabilisation.

Definition 1.19 (BKZ-reduced basis)

Let $B = \{b_1, \cdots, b_n\}$ be a basis of a lattice in $\mathbf{R}^n$. Let b_i^* denote the vectors obtained from Gram–Schmidt orthogonalisation (see Definition 1.3.2). The basis B is said to be (η, δ, B_0)-*BKZ-reduced* when $1/4 < \eta < 1$ and $\frac{1}{2} \leqslant \eta < \sqrt{\delta}$ and

† B is (η, δ)-LLL-reduced.

† For every integer $i \in \{1, \cdots, n-1\}$,

$$\|b_i^*\| = \lambda_i\big(\mathscr{L}(b_i, b_{i+1}, \cdots, b_k)\big) \quad \text{with } k := \min(i + B_0 - 1, n).$$

Definition 1.20 (BKZ algorithm)

The Blockwise-Korkine-Zolotarev (BKZ) algorithm is the best known practical method for obtaining a basis that is more strongly reduced than one produced by LLL. It produces an (η, δ, B_0)-BKZ-reduced basis, with block size $B_0 \geqslant 2$, from an initial basis $B = (b_1, \cdots, b_n)$ of a lattice $\mathscr{L}$.

BKZ Algorithm

Input: a basis $B = (b_1, \cdots, b_n)$, a block size $B_0 \in \{2, \cdots, n\}$.
Output: B, which is (η, δ, B_0)-BKZ-reduced.
$\ell \longleftarrow 0$, $j \longleftarrow 0$
Apply LLL to B to initialise the basis
While $\ell < n - 1$ **do**
    $j \longleftarrow (j \bmod (n-1)) + 1$, $k \longleftarrow \min(j + B_0 - 1, n)$, $h \longleftarrow \min(k + 1, n)$ ▷ *Definition of the block*
    Use an SVP oracle (for example, Enum) to find a short vector v in this block
    If $v \neq (1, 0, \cdots, 0)$ **then**
        $\ell \longleftarrow 0$
        Insert v into the basis at index j (replacing b_j with v and removing dependencies within the block)
        Apply (η, δ)-LLL to the block $(b_j, \ldots, b_h)$.
    Else
        $\ell \longleftarrow \ell + 1$
        Apply (η, δ)-LLL to $(b_1, \ldots, b_h)$
    End If
End While
Return B

Remark. In post-quantum cryptography, BKZ with a suitably chosen parameter B_0 (B_0 close to 1) is used to estimate the security of LWE/NTRU schemes. BKZ is also used in estimators such as the LWE Estimator

to determine the security level, in bits, of post-quantum schemes. One sets $\eta = 1/2$, or slightly larger, for numerical stability.

To conclude this section, the following table compares the LLL and BKZ algorithms:

Property	**LLL**	**BKZ**
Complexity	Polynomial	Exponential in B_0
Reduction quality	Poor	Good for large B_0
Use in cryptography	Limited	Standard for LWE/NTRU
Approach	Weak global reduction	Strong local reduction (blocks)

Comparison of LLL and BKZ for lattice reduction

2 The SIS and LWE Problems

Lattice-based cryptography relies on mathematical problems that are hard even for a quantum computer. Among the most important are the SIS (Short Integer Solution) and LWE (Learning With Errors) problems, introduced by Ajtai in 1996 and Regev in 2005, respectively. These problems are central to the design of cryptographic primitives resistant to quantum attacks.

In what follows, n and m denote two natural numbers greater than or equal to 1, and q is a prime number. We write $\mathbf{Z}_q := \mathbf{Z}/q\mathbf{Z}$.

2.1 The SIS Problem

Definition 2.1 (Homogeneous SIS problem)

Given a matrix A sampled uniformly from $\mathscr{M}_{n,m}(\mathbf{Z})$, which we denote by

$$A \longleftarrow \mathscr{U}(\mathscr{M}_{n,m}(\mathbf{Z}_q)),$$

the SIS problem consists of finding a *small* non-zero vector $z \in \mathbf{Z}^m$ such that $Az = 0 \pmod q$. We denote this problem by $\mathrm{SIS}(n, m, q)$.

Remarks.

† In practice, the prime number q is of order 10^4, and $\|z\|_\infty \ll \frac{q}{2}$.

† If $n > m$, then the equation $Az = 0 \pmod q$ in the unknown $z \in \mathbf{Z}^m$ has the unique solution $z = 0$. In this case, no SIS solution exists. From now on, we therefore assume that $n < m$, which guarantees the existence of infinitely many solutions.

† The SIS solution is not unique: indeed, if z is a solution, then $-z \pmod q$ is also a solution.

Definition 2.2 (Inhomogeneous SIS problem)

Let $A \longleftarrow \mathscr{U}(\mathscr{M}_{n,m}(\mathbf{Z}_q))$ and let $b \longleftarrow \mathscr{U}(\mathbf{Z}_q^n)$ be a vector. The ISIS problem consists of finding a *small* vector $z \in \mathbf{Z}^m$ such that $Az = b \pmod q$. We denote this problem by $\mathrm{ISIS}(n, m, q)$.

Definition 2.3

Let P and Q be two decision problems. We say that P is polynomial-time reducible to Q, denoted by $P \leqslant_p Q$, if there exists a polynomial-time computable function f such that, for every x:

$$x \in P \quad \Longleftrightarrow \quad f(x) \in Q.$$

Proposition 2.4

$\mathrm{SIS} \leqslant_p \mathrm{ISIS}$.

Proof. We shall exhibit a reduction function

$$f : \mathscr{M}_{n,m}(\mathbf{Z}_q) \longrightarrow \mathscr{M}_{n,m-1}(\mathbf{Z}_q) \times \mathbf{Z}_q^n$$

and show that it satisfies the definition.

Let A be viewed as a matrix A' with $m-1$ columns adjoining a column vector $-b'$:

$$A = (A' | -b') \longleftarrow \mathscr{U}\left(\mathscr{M}_{n,m}(\mathbf{Z}_q)\right),$$

with $A' \longleftarrow \mathscr{U}(\mathscr{M}_{n,m-1}(\mathbf{Z}_q))$ and $b' \longleftarrow \mathscr{U}(\mathbf{Z}_q^n)$. We then set

$$f(A) = (A', b'),$$

which is an instance of ISIS. Clearly, f is computable in polynomial time (by extracting the last column).

It remains to show that, for every instance A:

$$A \in \text{SIS} \quad \Longleftrightarrow \quad f(A) \in \text{ISIS}.$$

$\boxed{\Longrightarrow}$ If $A \in$ SIS, then there exists a non-zero vector $z = (z', z_m) \in \mathbf{Z}^{m-1} \times \mathbf{Z}$ such that

$$Az = 0 \pmod q.$$

This condition can be written as

$$A'z' - b'z_m = 0 \pmod q,$$

i.e.

$$A'z' = b'z_m \pmod q.$$

If $z_m \not\equiv 0 \pmod q$, define $y = (z_m)^{-1}z'$, which is non-zero; then $A'y = b' \pmod q$. If $z_m = 0 \pmod q$, then $z' \neq 0$ and $A'z' = 0$, which also gives a non-trivial solution to ISIS. In both cases, $(A', b') \in$ ISIS.

$\boxed{\Longleftarrow}$ If $(A', b') \in$ ISIS, there exists $z' \in \mathbf{Z}^{m-1}\backslash\{0\}$ such that

$$A'z' = b' \pmod q.$$

We then take $z = \begin{pmatrix} z' \\ 1 \end{pmatrix}$, which is non-zero, and

$$Az = (A' | -b') \begin{pmatrix} z' \\ 1 \end{pmatrix} = A'z' - b' = 0 \pmod q.$$

Thus z is a non-trivial solution to the SIS instance A, and therefore $A \in$ SIS.

The function f is computable in polynomial time and satisfies

$$A \in \text{SIS} \quad \Longleftrightarrow \quad f(A) \in \text{ISIS},$$

hence SIS $\leqslant_p$ ISIS. □

Proposition 2.5
ISIS $\leqslant_p$ SIS.

Proof. Let (A, b) be an instance of the ISIS problem with

$$A \longleftarrow \mathscr{U}(\mathscr{M}_{n,m}(\mathbf{Z}_q)), \quad b \longleftarrow \mathscr{U}(\mathbf{Z}_q^n).$$

Let

$$j \in \{1, \cdots, m+1\} \quad \text{and} \quad c \in \mathbf{Z}\backslash\{0\}.$$

Construct the matrix

$$A' = \left(A | -c^{-1}b\right) \in \mathscr{M}_{n,m+1}(\mathbf{Z}_q),$$

where the vector $-c^{-1}b$ has been inserted as the j-th column of A. Apply an algorithm for solving SIS to the instance $(A', 0)$ to obtain a vector

$$z' \in \mathbf{Z}^{m+1} \quad \text{such that} \quad A'z' = 0 \pmod q.$$

If the j-th coordinate of z' is equal to c, then deleting this coordinate defines

$$z = \left(z'_1, \cdots, z'_{j-1}, z'_{j+1}, \cdots, z'_{m+1}\right) \in \mathbf{Z}^m$$

which satisfies $Az = b \pmod q$. Thus, z is a solution to the ISIS instance (A, b) constructed from the SIS algorithm. □

2.2 The LWE Problem

We now turn to the LWE problem, which first requires a brief review of the notion of *indistinguishability*. Two probability distributions are said to be *computationally indistinguishable* if no efficient algorithm, running in at most polynomial time, can distinguish them; that is, given access to samples from one distribution, it cannot determine which distribution they come from.

We formalise this notion by considering a *distinguisher*, which is simply a probabilistic algorithm placed in the context of two games:

Definition 2.6 (Distinguisher)

Let D_0, D_1 be two distributions over $\{0,1\}^n$. We define two games G_0 and G_1 between an adversary $\mathscr{A}$ and a challenger $\mathscr{C}$ as follows:

1. During the game, $\mathscr{A}$ may submit one or more queries to $\mathscr{C}$.
2. For each query, $\mathscr{C}$ returns an element x sampled according to the distribution D_b, where $b = 0$ in game G_0 and $b = 1$ in game G_1.
3. At the end, $\mathscr{A}$ returns a bit b' to $\mathscr{C}$, corresponding to its guess of the distribution D_b used by $\mathscr{C}$.

The objective of $\mathscr{A}$ is to distinguish whether $\mathscr{C}$ is playing game G_0 or game G_1 (that is, whether it samples from D_0 or D_1).

The *advantage* of $\mathscr{A}$ is defined by

$$\mathrm{Avt}(\mathscr{A}) := \left|\mathbf{P}(\mathscr{A} \xrightarrow{G_0} 1) - \mathbf{P}(\mathscr{A} \xrightarrow{G_1} 1)\right|.$$

$\mathscr{A}$ is a *distinguisher* if there exists a non-negligible function $\varepsilon > 0$[1] in n such that $\mathrm{Avt}(\mathscr{A}) \geqslant \varepsilon(n)$.
In other words, $\mathscr{A}$ is able to distinguish between G_0 and G_1.

An alternative definition uses a single game:

Definition 2.7

Let D_0, D_1 be two distributions over $\{0,1\}^n$. We define a game G between the adversary $\mathscr{A}$ and the challenger $\mathscr{C}$ that proceeds in three phases:

1. The challenger uniformly samples a bit $b \longleftarrow \mathscr{U}(\{0,1\})$.
2. Upon a query from the adversary, the challenger returns an element sampled according to the distribution D_b, i.e. $x \longleftarrow D_b(\{0,1\}^n)$ (this may be repeated any number of times).
3. $\mathscr{A}$ returns a bit b' to $\mathscr{C}$. If $b = b'$, $\mathscr{C}$ returns "Win"; otherwise, it returns "Lose" to $\mathscr{A}$.

$\mathscr{A}$ is a *distinguisher* if there exists a non-negligible function ε in n such that

$$\mathbf{P}(\mathrm{Win}) \geqslant \tfrac{1}{2} + \varepsilon(n).$$

Remark. The adversary $\mathscr{A}$ may make as many queries as it wishes, but there is of course a limit: it must be a polynomial-time algorithm. We say that $\mathscr{A}$ is PPT (*probabilistic polynomial time*).

Definition 2.8 (Computational indistinguishability)

Two distributions D_0 and D_1 are computationally indistinguishable if, for every PPT distinguisher $\mathscr{D}$, there exists a negligible function ε such that

$$\mathrm{Avt}(\mathscr{D}) := \left|\mathbf{P}(\mathscr{D} \xrightarrow{G_0} 1) - \mathbf{P}(\mathscr{D} \xrightarrow{G_1} 1)\right| \leqslant \varepsilon(n).$$

Equivalently, for every PPT distinguisher $\mathscr{D}$, there exists a negligible function ε such that

$$\left|\mathbf{P}(\mathrm{Win}) - \tfrac{1}{2}\right| \leqslant \varepsilon(n).$$

[1] For every polynomial P, there exists λ_0 such that, for every $\lambda > \lambda_0$, we have $\varepsilon(\lambda) < \frac{1}{P(\lambda)}$.

Proof. We prove the equivalence of these two definitions by means of two reductions. In each direction, we assume the existence of a distinguisher, and hence an algorithm, for one definition and use it to construct a distinguisher, and hence another algorithm, for the other definition.

Let $\mathscr{A}$ be a distinguisher according to Definition 2.7. Let $\mathscr{A}'$ act exactly like $\mathscr{A}$, except that it returns b' and terminates instead of sending b' to the challenger. Then:

$$\begin{aligned}\mathrm{Avt}(\mathscr{A}') &= \mathbf{P}(b'=1 \mid b=0) - \mathbf{P}(b'=1 \mid b=1)\\ &= \big|1 - \mathbf{P}(b'=0 \mid b=0) - \mathbf{P}(b'=1 \mid b=1)\big|\\ &= \big|1 - 2\mathbf{P}(b'=0 \text{ and } b=0) - 2\mathbf{P}(b'=1 \text{ and } b=1)\big|\\ &= \big|1 - 2\mathbf{P}(\mathrm{Win})\big| \geqslant 2\varepsilon.\end{aligned}$$

Conversely, consider a distinguisher $\mathscr{A}'$ according to Definition 2.6. Let $\mathscr{A}$ act like $\mathscr{A}'$, but send b' to the challenger. Then:

$$\begin{aligned}\mathbf{P}(\mathrm{Win}) &= \mathbf{P}(b'=0 \text{ and } b=0) + \mathbf{P}(b'=1 \text{ and } b=1)\\ &= \tfrac{1}{2}\big(\mathbf{P}(b'=0 \mid b=0) + \mathbf{P}(b'=1 \mid b=1)\big)\\ &= \tfrac{1}{2}\big(1 + \mathrm{Avt}(\mathscr{A}')\big).\end{aligned}$$

Assuming that $\mathbf{P}(b'=1 \mid b=1) \geqslant \mathbf{P}(b'=1 \mid b=0)$ (otherwise $\mathscr{A}'$ returns $1-b'$ instead of b'), we obtain

$$\mathbf{P}(\mathrm{Win}) \geqslant \tfrac{1}{2} + \tfrac{\varepsilon}{2}.$$

□

Remark. The factor 1/2 separating the two definitions leads some authors to use

$$2\big|\mathbf{P}(\mathrm{Win}) - \tfrac{1}{2}\big|$$

instead of

$$\big|\mathbf{P}(\mathrm{Win}) - \tfrac{1}{2}\big|.$$

Here, however, the definition is equivalent because

$$2\varepsilon(n) = \varepsilon(n).$$

Definition 2.9 (Statistical distance)
Let X, Y, and Z be three random variables over a countable set A. The *statistical distance* between X and Y is defined by:

$$\Delta(X,Y) = \tfrac{1}{2}\sum_{a\in A}\big|\mathbf{P}(X=a) - \mathbf{P}(Y=a)\big|.$$

It is straightforward to verify that this is indeed a distance:

† $\Delta(X,Y) \geqslant 0$ and equals 0 if and only if X and Y have the same distribution,

† $\Delta(X,Y) = \Delta(Y,X)$,

† $\Delta(X,Z) \leqslant \Delta(X,Y) + \Delta(Y,Z)$.

Definition 2.10 (Statistical indistinguishability)
Two distributions D_0, D_1 are *statistically indistinguishable* if there exists a negligible function ε such that:

$$\Delta(D_0, D_1) \leqslant \varepsilon(n).$$

Remark. This is a stronger property than computational indistinguishability.

We can now define the LWE distribution.

Definition 2.11 (LWE distribution)

We consider the following discrete Gaussian distribution, defined over $\mathbf{Z}$ and given for each integer $x \in \mathbf{Z}$ by:

$$D_{\sigma,c}(x) = \frac{\exp\left(-\pi(x-c)^2/\sigma^2\right)}{\sum_{k\in\mathbf{Z}} \exp\left(-\pi(k-c)^2/\sigma^2\right)}.$$

This distribution can be sampled in quasi-linear time[2] and is used to construct the following object:

Let $n \geqslant 1$ be an integer, let $q \geqslant 2$ be prime, let $\alpha \in [0,1]$ be a real parameter, and let $s \in \mathbf{Z}_q^n$ be a vector. The *LWE distribution* (Learning With Errors), denoted by $D_{n,q,\alpha}(s)$, is defined over $\mathbf{Z}_q^n \times \mathbf{Z}_q$ as follows:

† Sample the "error term" $e \longleftarrow D_{\alpha q,0}$.

† Sample $a \longleftarrow \mathscr{U}(\mathbf{Z}_q^n)$ uniformly.

† Return the pair $(a, \langle a, s\rangle + e \bmod q)$.

Definition 2.12 (LWE problem)

This distribution makes it possible to define LWE either as a search problem or as a decision problem:

† *The search LWE problem (denoted RLWE)*: given samples distributed according to $D_{n,q,\alpha}(s)$, recover s.

† *The decision LWE problem (denoted DLWE)*: if $s \in \mathbf{Z}_q^n$ is sampled uniformly, distinguish the distribution $D_{n,q,\alpha}(s)$ from the uniform distribution $\mathscr{U}(\mathbf{Z}_q^n \times \mathbf{Z}_q)$.

Remarks.

† Most often, q is chosen to be polynomial in n, and α lies between $1/q$ and 1.

† If α is close to 0, then, with probability very close to 1, the error e in a sample is 0. In this case, n samples yield a linear system in s that can be solved in polynomial time.

† If α is close to 1, then e is almost uniformly distributed modulo q. The distribution $D_{n,q,\alpha}(s)$ is then close to the uniform distribution, independently of s, and recovering s becomes impossible.

† The RLWE and DLWE problems can be reformulated more simply as follows:

- RLWE: Find $s \in \mathbf{Z}_q^n$, given $A \in \mathscr{M}_{m,n}(\mathbf{Z}_q)$ and $b = As + e \bmod q$, where e is Gaussian.
- DLWE: Distinguish between uniformly random (A, b) and $(A, As + e \bmod q)$, where

$$A \longleftarrow \mathscr{U}(\mathscr{M}_{m,n}(\mathbf{Z}_q)), \quad s \longleftarrow \mathscr{U}(\mathbf{Z}_q^n) \text{ is secret}, \quad e \text{ is Gaussian.}$$

Proposition 2.13 (Equivalence of RLWE and DLWE)

When q is polynomial in n, the problems RLWE and DLWE are equivalent under polynomial-time reductions.

Proof.

DLWE $\leqslant_p$ RLWE Suppose that we have an algorithm A that solves the search problem, and use A to solve the decision problem. As input to the decision problem, we receive a set of samples a_i, b_i for which either

$$b_i = a_i \cdot s + e_i,$$

or b_i is uniform and independent of a_i. Our proof strategy is as follows. We give these samples to the algorithm A, which returns a value s. We then test whether $b - As$ is short, that is, whether it has "small" norm. If so, we conclude that the samples are LWE samples; otherwise, we classify them as random. This procedure works. First consider the LWE case. The search algorithm succeeds, and recomputing $b - As$ indeed yields a short vector, so the LWE case is correctly identified. Now consider the random case. The search algorithm fails and returns some vector s. By computing $b - As$, we obtain a uniform vector in $\mathbf{Z}_q^n$. The probability that this vector is short is negligible. Hence, our procedure succeeds with probability close to 1.

[2] In other words, if N denotes the input size (for example, the number of bits required to describe the distribution parameters), then a quasi-linear algorithm runs in $O(N \cdot (\log N)^c)$ for some constant exponent c.

RLWE $\leqslant_p$ DLWE We seek to recover the first coordinate of s by testing whether it is equal to a chosen value k. Since q is polynomial in n, it suffices to repeat this operation for every k, and then for every coordinate, thereby reconstructing s in full. An important point is that the decision routine must be reliable, because it will be called several times. To reduce its error probability, we run it repeatedly and take the majority output. From now on, we assume that the decision routine succeeds on every call. To use the routine, we transform the samples so that, if our guess k is correct, they remain LWE samples, whereas otherwise they become uniform. To do this, we inject our own randomness into each sample. For each LWE sample $(a_i, b_i = a_i \cdot s + e_i)$, we proceed as follows: sample r uniformly from $\mathbf{Z}_q$, add $(r, 0, \cdots, 0)$ to a_i, and add (rk) to b_i. Then:

† If $s_0 = k$, then the modified sample is

$$\big(a'_i, a'_i \cdot s + e_i\big) \quad \text{where} \quad a'_i = a_i + (r, 0, \cdots, 0),$$

which remains uniformly random (an essential requirement for the decision procedure).

† If $s_0 \neq k$, then the modified sample is

$$\big(a'_i, b'_i\big) \quad \text{with} \quad b'_i = a_i \cdot s + e_i + rk = a'_i \cdot s + r(k - s_0) + e_i.$$

Here, k and s_0 are constants, so $r(k - s_0)$ is uniformly random in $\mathbf{Z}_q$ and independent of a'_i. The new samples (a'_i, b'_i) are therefore both uniform and uncorrelated.

Thus, the decision routine distinguishes the two cases. Finally, because the decision LWE problem assumes $s \longleftarrow \mathscr{U}(\mathbf{Z}_q^n)$, it remains to show that recovering a random s allows one to recover an arbitrary s. Indeed, for a fixed s, sample $t \longleftarrow \mathscr{U}(\mathbf{Z}_q^n)$ and transform each sample

$$(a, a \cdot s + e) \quad \longmapsto \quad \big(a, a \cdot (s + t) + e\big).$$

The new secret $s + t$ is uniform; after recovering it, we deduce s. □

Remark. From an algorithmic perspective, LWE appears to be extremely difficult to solve. The best known algorithm for general parameters (n, q, α) is based on lattice reduction. It is Schnorr's BKZ algorithm, which improves upon the LLL algorithm. This connection with lattices is not accidental. Lattice algorithms provide the best attacks against LWE; conversely, one can show that if an efficient algorithm for solving LWE existed, then efficient algorithms would also exist for hard lattice problems.

More precisely, the existence of an efficient algorithm solving LWE with Gaussian error implies the existence of an efficient algorithm for solving GapSVP_γ:

Theorem 2.14
Let $q \geqslant 2$ and $\alpha \in]0, 1[$ be functions of n such that $\alpha q \geqslant 2\sqrt{n}$. If q is prime and polynomial in n, there exists a polynomial-time quantum reduction from GapSVP_γ in dimension n to $\mathrm{LWE}_{n,q,\alpha}$ with $\gamma = \widetilde{O}(n/\alpha)$. For every q, there exists a polynomial-time classical reduction from GapSVP_γ in dimension $\Theta(\sqrt{n})$ to $\mathrm{LWE}_{n,q,\alpha}$ with $\gamma = \widetilde{O}(n^2/\alpha)$.

2.3 Regev Encryption and IND-(CPA, CCA) Security

Encryption (or *public-key cryptography*) is a technique that enables a sender to transmit a confidential message securely to a recipient, even over an insecure channel.

- The primary purpose of encryption is confidentiality.
- It generally uses a public key, accessible to everyone, to encrypt the message, and a private key, kept secret by the recipient, to decrypt it.
- Example: if Alice wishes to send a secret message to Bob, she encrypts it using Bob's public key, and Bob decrypts it using his private key.

The first encryption scheme whose security relies on the hardness of LWE was proposed by Regev in 2005. The dimension n is its security parameter. Let n, ℓ, q be integers, with q prime, $\ell \geqslant 4(n+1)\log_2 q$, and $\alpha \in]0, 1/(8\ell)[$.

Because LWE introduces errors, we need a way to remove them during decryption. We formalise this using a pair of functions, Compress and Decompress, which will be used below. Compress takes an integer modulo q and decodes it as 0 if it is closer to 0, or as 1 if it is closer to $\lfloor q/2 \rfloor$. Decompress performs the inverse operation: it encodes 0 as 0 and 1 as $\lfloor q/2 \rfloor$.

$$\mathsf{Compress} : \left| \begin{array}{ccc} \mathbf{Z}_q & \longrightarrow & \{0,1\} \\ u & \longmapsto & \mathsf{round}\left(\frac{u}{\lfloor q/2 \rfloor}\right) \pmod 2 \end{array} \right.$$

$$\mathsf{Decompress} : \left| \begin{array}{ccc} \{0,1\} & \longrightarrow & \mathbf{Z}_q \\ m & \longmapsto & m\lfloor q/2 \rfloor \end{array} \right.$$

Compress and Decompress operate modulo q; from this perspective, any value close to q (for example, $q-1$ or $q-2$) is "small". For example, $\mathrm{Compress}(q-1) = 0$. More generally, these numbers modulo q should be viewed as represented by an integer between $-\lfloor q/2 \rfloor$ and $\lfloor q/2 \rfloor$.

We can now introduce Regev's public-key encryption scheme. It is secure but inefficient, because it encrypts only a single bit and requires a key of size $\mathcal{O}(n^2)$.

LWE PKE (Learning With Errors Public Key Encryption)

KeyGen:

- Private key: a random $s \in \mathbf{Z}_q^n$
- Public key: $(A, b := As + e)$, where A is a random matrix $A \in \mathbf{Z}_q^{\ell \times n}$ and $e \in \mathbf{Z}_q^\ell$ is sampled from the "small" error distribution (i.e. a discrete Gaussian)

Enc $m \in \{0,1\}$:

- Choose a random vector $r \in \{0,1\}^\ell$
- Return $c_1, c_2 := rA, \big(\mathsf{Decompress}(m) + r \cdot b\big)$

Dec $c = (c_1, c_2) \in \mathbf{Z}_q^{n+1}$:

- $m = \mathsf{Compress}\big(c_2 - c_1 \cdot s\big)$

To analyse decryption, compute:

$$\begin{aligned} c_2 - c_1 \cdot s &= \big(\mathsf{Decompress}(m) + r \cdot b\big) - (rA) \cdot s \\ &= \mathsf{Decompress}(m) + r \cdot (As + e) - rAs \\ &= \mathsf{Decompress}(m) + \underbrace{r \cdot e}_{\text{small}} \end{aligned}$$

We chose r uniformly from $\{0,1\}^\ell$, and e follows a discrete Gaussian distribution with parameter $\alpha q \leqslant q/(8m)$; therefore, with probability close to 1:

$$\left| \sum_i r_i e_i \right| \leqslant \|r\| \cdot \|e\| \leqslant \sqrt{m} \frac{q}{8\sqrt{m}} = \frac{q}{8}.$$

Applying Compress to this result should therefore recover the value of m. However, depending on the chosen error distribution, there may be a non-zero probability that the error is too large and decryption fails. Such failure probabilities also occur in modern schemes such as Kyber. It is sufficient to ensure that this probability is low enough never to arise in practice.

The proof of the *security* of Regev's encryption scheme relies on the following lemma, which we accept without proof and which is a special case of the *leftover hash lemma* (this lemma states that if an adversary obtains partial information about an n-bit value, for example t bits of information, then it is still possible to extract $n - t$ bits about which the adversary has no information).

Lemma 2.15
Let $\ell, n, q \geqslant 1$ be integers such that $\ell \geqslant 4n \log q$, with q prime, and let

$$A \longleftarrow \mathscr{U}\left(\mathbf{Z}_q^{\ell \times n}\right), \quad r \longleftarrow \mathscr{U}\left(\{0,1\}^{\ell}\right).$$

Then (A, rA) has statistical distance at most 2^{-n} from the uniform distribution over $\mathbf{Z}_q^{\ell \times n} \times \mathbf{Z}_q^n$.

Definition 2.16 (IND-CPA encryption scheme)
A public-key encryption scheme $(\mathsf{Gen}, \mathsf{Enc}, \mathsf{Dec})$ is said to be *indistinguishable under chosen-plaintext attack* (or IND-CPA) if no probabilistic polynomial-time adversary can distinguish, with probability significantly greater than $1/2$, the encryptions of two messages of its choice, even after being allowed to query an encryption oracle.

Formally, consider the following game between an adversary $\mathscr{A}$ and a challenger:

1. The challenger runs $(pk, sk) \longleftarrow \mathsf{Gen}(\lambda)$, where λ is a security parameter.
2. The adversary $\mathscr{A}$ receives the public key pk and has access to an encryption oracle $\mathsf{Enc}_{pk}(\cdot)$.
3. The adversary chooses two equal-length messages $m_0, m_1 \in \mathscr{M}$ and sends them to the challenger.
4. The challenger uniformly samples a bit $b \longleftarrow \mathscr{U}(\{0,1\})$, computes $c^* \longleftarrow \mathsf{Enc}_{pk}(m_b)$, and sends c^* to $\mathscr{A}$.
5. The adversary may continue to query the encryption oracle (without requesting the encryption of m_0 or m_1).
6. Finally, $\mathscr{A}$ outputs a bit $b' \in \{0,1\}$.

The scheme is IND-CPA secure if, for every PPT adversary $\mathscr{A}$, there exists a negligible function $\varepsilon(\lambda)$ such that:

$$\left| \mathbf{P}(b' = b) - \frac{1}{2} \right| \leqslant \varepsilon(\lambda)$$

In other words, even after seeing encryptions of messages of its choice, the adversary cannot guess b correctly with probability significantly better than random guessing.

Remarks.

† During the IND-CPA game, the adversary, which has the public key, can encrypt any message of its choice (hence the notion of "chosen plaintext").

† Encryption must necessarily be probabilistic; otherwise, there is a trivial attack in which the adversary encrypts the messages m_0 and m_1 itself.

† This notion also implies that the same key may be used multiple times.

We now define the notion of an IND-CCA encryption scheme. CCA security allows the adversary to perform not only encryptions but also decryptions, by granting access to a decryption oracle. Since the adversary does not possess the secret key, it must submit *decryption queries* to the challenger. However, if it were allowed to decrypt any ciphertext, the definition would be meaningless: it could decrypt the challenge ciphertext $c^* = \mathsf{Enc}(\mathsf{pk}, m_b)$. Decryption of this specific ciphertext is therefore prohibited. This gives security against *chosen-ciphertext attacks* (CCA, *chosen ciphertext attack*).

Definition 2.17 (IND-CCA encryption scheme)
The CCA security game is defined as follows:

1. The challenger $\mathscr{C}$ chooses a bit $b \longleftarrow \mathscr{U}(\{0,1\})$ and generates $(\mathsf{pk}, \mathsf{sk}) \longleftarrow \mathsf{KeyGen}(1^n)$. It sends pk to the adversary $\mathscr{A}$.
2. $\mathscr{A}$ chooses two messages m_0, m_1 and sends them to $\mathscr{C}$, which responds with $c^* = \mathsf{Enc}(\mathsf{pk}, m_b)$.
3. $\mathscr{A}$ returns a bit b'; it wins if $b' = b$.

During the game, $\mathscr{A}$ may submit decryption queries to the challenger: for any query on a ciphertext c, if $c \neq c^*$, then $\mathscr{C}$ returns $\mathsf{Dec}(\mathsf{sk}, c)$; otherwise, it returns nothing.

- In the IND-CCA1 game (*non-adaptive*), $\mathscr{A}$ may submit these queries only during Step 2, before learning c^*.
- In the IND-CCA2 game (*adaptive*), $\mathscr{A}$ may submit these queries at any time.

The advantage of $\mathscr{A}$ is defined by

$$\mathrm{Avt}^{\mathrm{CCA}}(\mathscr{A}) = \left|\mathbf{P}(\mathscr{A} \text{ wins}) - \tfrac{1}{2}\right|.$$

If, for every PPT adversary $\mathscr{A}$, $\mathrm{Avt}^{\mathrm{CCA}}(\mathscr{A})$ is negligible in n, then the scheme is said to be *IND-CCA(1,2)* secure.

Lemma 2.18
If LWE is NP-hard, Regev's scheme is IND-CPA secure.

Proof. We show that every PPT adversary $\mathscr{A}$ attacking the IND-CPA property of Regev's scheme has negligible advantage. The proof uses three games, G_0, G_1, and G_2, where G_0 is the IND-CPA game. We show that a PPT adversary cannot distinguish G_0 from G_1, nor G_1 from G_2, except with negligible probability, and then show that its advantage in game G_2 is zero. This establishes the result.

Algorithm: Game G_0 (IND-CPA)

1: $\mathsf{pk} \longleftarrow \mathsf{KeyGen}(n)$, i.e. $\mathsf{pk} = (A, b = As + e)$ sampled according to the LWE distribution.
2: $(m_0, m_1) \longleftarrow \mathscr{A}(\mathsf{pk})$.
3: $b' \longleftarrow U(\{0,1\})$.
4: $C^* \longleftarrow \mathsf{Enc}(\mathsf{pk}, m_{b'})$, i.e. $C^* = (rA, r \cdot b + \lfloor q/2 \rfloor m_{b'})$, where r is sampled uniformly.
5: $b'' \longleftarrow \mathscr{A}(C^*)$.
6: Return 1 if $b' = b''$, and 0 otherwise.

The advantage of $\mathscr{A}$ in game G_0 is

$$\mathrm{Avt}^{\text{IND-CPA}}(\mathscr{A}) = \left|\mathbf{P}(b'' = b') - \tfrac{1}{2}\right| = \left|\mathbf{P}(\mathscr{A} \text{ win in } G_0) - \tfrac{1}{2}\right|.$$

For game G_0, we modify public-key generation in G_0. Instead of the genuine LWE distribution, the second component is made uniform:

$$b \longleftarrow \mathscr{U}\left(\mathbf{Z}_q^n\right).$$

The public key therefore becomes $\mathsf{pk} = (A, b)$, where $A \in \mathbf{Z}_q^{\ell \times n}$ is sampled according to the LWE distribution and b is uniform over $\mathbf{Z}_q^n$.

Algorithm: Game G_1

1: $\mathsf{pk} = (A, b)$ where $A \longleftarrow \mathscr{U}\left(\mathbf{Z}_q^{\ell \times n}\right)$ and $b \longleftarrow \mathscr{U}\left(\mathbf{Z}_q^n\right)$.
2: $(m_0, m_1) \longleftarrow \mathscr{A}(\mathsf{pk})$.
3: $b' \longleftarrow U(\{0,1\})$.
4: $C^* \longleftarrow \mathsf{Enc}(\mathsf{pk}, m_{b'})$, i.e. $C^* = (rA, r \cdot b + \lfloor q/2 \rfloor m_{b'})$ where r is sampled uniformly.
5: $b'' \longleftarrow \mathscr{A}(C^*)$.
6: Return 1 if $b' = b''$, and 0 otherwise.

Under the decision-LWE assumption, games G_0 and G_1 are indistinguishable. In other words, the difference between the probabilities that the attacker $\mathscr{A}$ returns 1 in the two games is negligible:

$$\left|\mathbf{P}(\mathscr{A} \text{ win in } G_0) - \mathbf{P}(\mathscr{A} \text{ win in } G_1)\right| = \varepsilon(n).$$

Otherwise, $\mathscr{A}$ could be used as a distinguisher for LWE.

For game G_2, we modify the generation of the challenge ciphertext in G_1. We replace

$$C^* = \left(rA, r \cdot b + \lfloor q/2 \rfloor m_{b'}\right)$$

with a random value:

$$C^* \longleftarrow \mathscr{U}\left(\mathbf{Z}_q^{n+1}\right).$$

Algorithm: Game G_2

1: $\mathsf{pk} = (A, b)$ where $A \longleftarrow \mathscr{U}\left(\mathbf{Z}_q^{\ell \times n}\right)$, and $b \longleftarrow \mathscr{U}\left(\mathbf{Z}_q^n\right)$.
2: $(m_0, m_1) \longleftarrow \mathscr{A}(\mathsf{pk})$.
3: $b' \longleftarrow \mathscr{U}(\{0,1\})$.
4: $C^* \longleftarrow \mathsf{Enc}(\mathsf{pk}, m_{b'})$, i.e. $C^* = \left(rA, r \cdot b + \lfloor q/2 \rfloor m_{b'}\right)$, where $r \longleftarrow \mathscr{U}(\{0,1\}^\ell)$.
5: $b'' \longleftarrow \mathscr{A}(C^*)$.
6: Return 1 if $b' = b''$, and 0 otherwise.

Consider what the adversary receives in the two cases. In G_1, we have already replaced b with a uniformly random value independent of A. Consequently, the ciphertext C^* is $\left(rA, u\right)$, where u and rA are uniform and independent. Moreover, the adversary knows A. In G_2, we have replaced C^* with a uniform value. This makes no difference to the right-hand component. For the left-hand component, the difference in probability is bounded by the statistical distance between (A, rA) and (A, U), which is itself negligible by Lemma 2.15. Consequently:

$$\left|\mathbf{P}_{G_1}(\mathscr{A} \text{ wins}) - \mathbf{P}_{G_2}(\mathscr{A} \text{ wins})\right| = \varepsilon(n).$$

Finally, in G_2, $\mathscr{A}$ receives a random challenge ciphertext. Consequently:

$$\mathbf{P}_{G_2}(\mathscr{A} \text{ wins}) = \tfrac{1}{2}.$$

We therefore conclude:

$$\begin{aligned}
\mathrm{Avt}^{\text{IND-CPA}}(\mathscr{A}) &= \left|\mathbf{P}_{G_0}(\mathscr{A} \text{ wins}) - \tfrac{1}{2}\right| \\
&\leqslant \underbrace{\left|\mathbf{P}_{G_0}(\mathscr{A} \text{ wins}) - \mathbf{P}_{G_1}(\mathscr{A} \text{ wins})\right|}_{\varepsilon(n) \text{ (LWE)}} \\
&\quad + \underbrace{\left|\mathbf{P}_{G_1}(\mathscr{A} \text{ wins}) - \mathbf{P}_{G_2}(\mathscr{A} \text{ wins})\right|}_{\varepsilon(n) \text{ (statistical)}} \\
&\quad + \underbrace{\left|\mathbf{P}_{G_2}(\mathscr{A} \text{ wins}) - \tfrac{1}{2}\right|}_{=0} \\
&\leqslant \varepsilon(n).
\end{aligned}$$

□

3 SIS and LWE Lattices

In this section, we examine the connection between the SIS/LWE problems and lattices.

3.1 The SIS Lattice

Let $n, m \in \mathbf{N}^*$ with $m > n$.

Proposition 3.1

Let $A \in \mathscr{M}_{n,m}(\mathbf{Z}_q)$. The set $\mathscr{L}_A^\perp := \left\{z \in \mathbf{Z}^m \mid Az = 0 \pmod q\right\}$ is a lattice in $\mathbf{R}^m$.

Proof. The set $\mathscr{L}_A^\perp$ is an additive subgroup of $\mathbf{R}^m$. Moreover, $\mathscr{L}_A^\perp$ is discrete as a subset of $\mathbf{Z}^m$, which is discrete in $\mathbf{R}^m$. Indeed, for every $z \in \mathbf{Z}^m$, we have

$$B\left(z, \frac{1}{3}\right) \cap \mathbf{Z}^m = \{z\},$$

where $B(z, \frac{1}{3}) := \{y \in \mathbf{R}^m \mid \|y - z\| < \frac{1}{3}\}$. □

Definition 3.2 (SIS lattice associated with A)

Let $A \in \mathscr{M}_{n,m}(\mathbf{Z}_q)$. The set $\mathscr{L}_A^\perp$ is called the *SIS lattice* associated with A, or the *dual lattice* associated with A.

Proposition 3.3 (Full rank of the SIS lattice)

Let $A \in \mathscr{M}_{n,m}(\mathbf{Z}_q)$. The SIS lattice associated with A has full rank m.

Proof. The lattice $q\mathbf{Z}^m$ is clearly contained in $\mathscr{L}_A^\perp$. The vectors

$$(q, 0, \cdots, 0), (0, q, 0, \cdots, 0), \cdots, (0, \cdots, 0, q)$$

belong to $q\mathbf{Z}^m$ and are linearly independent over $\mathbf{R}$. It follows that $q\mathbf{Z}^m$ is a lattice of rank m, and therefore so is $\mathscr{L}_A^\perp$. □

Remarks.

† $\mathscr{L}_A^\perp$ is a q-ary lattice, meaning that, for every $z \in \mathbf{Z}^m$, we have $z \in \mathscr{L}_A^\perp$ if and only if $z \bmod q \in \mathscr{L}_A^\perp$.

† A basis matrix of $q\mathbf{Z}^m$ is qI_m. Hence $\mathrm{vol}(q\mathbf{Z}^m) = |\det(qI_m)| = q^m$, and therefore $\mathrm{vol}(\mathscr{L}_A^\perp) \leqslant q^m$.

Proposition 3.4 (Volume of the SIS lattice)

Let $A \in \mathscr{M}_{n,m}(\mathbf{Z}_q)$ have rank $n < m$ over $\mathbf{Z}_q$. Then the lattice $\mathscr{L}_A^\perp$ satisfies

$$\mathrm{vol}(\mathscr{L}_A^\perp) = q^n.$$

Proof. The additive group $\mathbf{Z}^m$ and its subgroup $\mathscr{L}_A^\perp$ are free $\mathbf{Z}$-modules of the same rank m. Hence $\mathbf{Z}^m/\mathscr{L}_A^\perp$ is finite and, by Theorem 5.4 (in the appendix), we have

$$\mathrm{vol}(\mathscr{L}_A^\perp) = \left|\mathbf{Z}^m/\mathscr{L}_A^\perp\right|.$$

For $x, y \in \mathbf{Z}^m$,

$$\begin{aligned} \mathscr{L}_A^\perp + x = \mathscr{L}_A^\perp + y \quad &\iff \quad x - y \in \mathscr{L}_A^\perp \\ &\iff \quad A(x-y) = 0 \pmod q \\ &\iff \quad Ax = Ay \pmod q. \end{aligned}$$

Since A has rank n over $\mathbf{Z}_q$, the space generated by its family of column vectors in $\mathbf{Z}_q^n$ contains exactly q^n elements. There are therefore q^n cosets of $\mathscr{L}_A^\perp$ in $\mathbf{Z}^m$, whence $\left|\mathbf{Z}^m/\mathscr{L}_A^\perp\right| = q^n$ and $\mathrm{vol}(\mathscr{L}_A^\perp) = q^n$. □

Proposition 3.5 (Explicit basis of the SIS lattice)

Let $A \in \mathscr{M}_{n,m}(\mathbf{Z}_q)$. Assume that the first n columns of A are linearly independent over $\mathbf{Z}_q$. Then A can be transformed by elementary row operations (modulo q) into a matrix

$$\widetilde{A} = \left(I_n \mid \overline{A}\right) \quad \text{where} \quad \overline{A} \in \mathscr{M}_{n,m-n}(\mathbf{Z}_q).$$

The matrix

$$C = \begin{pmatrix} qI_n & -\overline{A} \\ 0 & I_{m-n} \end{pmatrix} \in \mathscr{M}_m(\mathbf{Z}_q)$$

is then a basis matrix of the lattice $\mathscr{L}_A^\perp$.

Proof. Since A and $\widetilde{A}$ are row-equivalent over $\mathbf{Z}_q$, they have the same kernel modulo q. In other words, $\ker(A) = \ker(\widetilde{A}) \pmod q$, and therefore $\mathscr{L}_A^\perp = \mathscr{L}_{\widetilde{A}}^\perp$. Every column v of C satisfies $\widetilde{A}v = 0 \pmod q$, hence $v \in \mathscr{L}_{\widetilde{A}}^\perp$.

Moreover, since $\det(C) = q^n$, the columns of C are linearly independent over $\mathbf{R}$. Thus, C is a basis matrix of a full-rank sublattice $\mathscr{L}$ of $\mathscr{L}_A^\perp$.

Finally, since $\mathrm{vol}(\mathscr{L}) = q^n = \mathrm{vol}(\mathscr{L}_A^\perp) = \mathrm{vol}(\mathscr{L}_{\widetilde{A}}^\perp)$, we have $\mathscr{L}_{\widetilde{A}}^\perp = \mathscr{L}$. Thus, C is a basis matrix of the SIS lattice $\mathscr{L}_A^\perp$. □

Remark. The SIS problem can therefore be reformulated in lattice terms as follows: given a matrix $A \in \mathscr{M}_{n,m}(\mathbf{Z}_q)$, SIS consists of finding a small non-zero vector $z \in \mathbf{Z}^m$ belonging to the lattice

$$\mathscr{L}_A^\perp = \mathscr{L}(C) \quad \subseteq \mathbf{Z}^m,$$

where

$$C = \begin{pmatrix} qI_n & -\overline{A} \\ 0 & I_{m-n} \end{pmatrix}$$

is a basis matrix of $\mathscr{L}_A^\perp$.

3.2 The LWE Lattice

In what follows, we consider the RLWE problem in which the error vector e is sampled from the uniform distribution: $e \longleftarrow \mathscr{U}(\mathbf{Z}_q^m)$.

Definition 3.6 (LWE lattice)
The *LWE lattice* is defined by

$$\mathscr{L}_A = \left\{ y \in \mathbf{Z}^m \mid Az = y \pmod{q} \text{ for some } z \in \mathbf{Z}^n \right\} \subseteq \mathbf{R}^m.$$

Proposition 3.7
$\mathscr{L}_A$ is a full-rank integral q-ary lattice in $\mathbf{R}^m$ (that is, a discrete additive subgroup of $\mathbf{R}^m$).

Proposition 3.8 (A basis of the LWE lattice)
Let

$$A = \begin{pmatrix} A_1 \\ A_2 \end{pmatrix}$$

where $A_1 \in \mathscr{M}_n(\mathbf{Z}_q)$ is invertible modulo q and $A_2 \in \mathscr{M}_{m-n,n}(\mathbf{Z}_q)$. Set

$$D_2 \equiv A_2 A_1^{-1} \pmod{q}, \quad D = \begin{pmatrix} I_n & 0 \\ D_2 & qI_{m-n} \end{pmatrix} \in \mathscr{M}_m(\mathbf{Z}).$$

Then D is a basis matrix of $\mathscr{L}_A$, and $\mathrm{vol}(\mathscr{L}_A) = q^{m-n}$.

Proof. We have $\det(D) = q^{m-n}$, so the columns of D are linearly independent over $\mathbf{R}$. Write a vector $y \in \mathbf{Z}^m$ as

$$y = \begin{pmatrix} y_1 \\ y_2 \end{pmatrix}, \quad y_1 \in \mathbf{Z}^n, \quad y_2 \in \mathbf{Z}^{m-n}.$$

Then

$$\begin{aligned} y \in \mathscr{L}_A &\iff Az \equiv y \pmod{q} \text{ for some } z \in \mathbf{Z}^n \\ &\iff \begin{cases} A_1 z \equiv y_1 \pmod{q} \\ A_2 z \equiv y_2 \pmod{q} \end{cases} \\ &\iff y_2 \equiv A_2 A_1^{-1} y_1 \pmod{q} \\ &\iff y_2 = D_2 y_1 + qc \text{ for some } c \in \mathbf{Z}^{m-n}. \end{aligned}$$

It follows that

$$y = D \begin{pmatrix} y_1 \\ c \end{pmatrix},$$

which shows that the columns of D generate $\mathscr{L}_A$. Since the columns of D are linearly independent over $\mathbf{R}$, this proves that D is a basis of $\mathscr{L}_A$. □

Remark. For an LWE instance (A, b, s, e), we have $y = As \bmod q \in \mathscr{L}_A$ and $\|b - y\|_2 = \|e\|_2 \leqslant \sqrt{m}q$, where m is the dimension of the lattice. Thus, LWE is a special case of the following lattice problem:

Definition 3.9 (Bounded Distance Decoding (BDD_α))
Let $\mathscr{L} = \mathscr{L}(D) \subseteq \mathbf{R}^m$ be a lattice given by a basis D, and let $b \in \mathbf{R}^m$ be such that there exists a unique vector $y \in \mathscr{L}$ satisfying

$$\|b - y\|_2 \leqslant \alpha.$$

The problem BDD_α consists of recovering this vector y.

Proposition 3.10
Let $\mathscr{L} = \mathscr{L}(D) \subseteq \mathbf{R}^m$ be a lattice and let $b \in \mathbf{R}^m$ be such that there exists a unique $y \in \mathscr{L}$ satisfying

$\|b - y\|_2 \leqslant \alpha$. Set

$$D' = \begin{pmatrix} D & -b \\ 0 & \alpha \end{pmatrix}, \quad \mathscr{L}' = \mathscr{L}(D') \subseteq \mathbf{R}^{m+1}.$$

Then solving SVP on $\mathscr{L}'$ makes it possible to solve the BDD_α instance $(\mathscr{L}, b)$.

Proof. Set

$$D' = \begin{pmatrix} D & -b \\ 0 & \alpha \end{pmatrix}, \quad \mathscr{L}' = \mathscr{L}(D') = \left\{ \begin{pmatrix} \nu - cb \\ c\alpha \end{pmatrix} \;\middle|\; \nu \in \mathscr{L}, c \in \mathbf{Z} \right\}.$$

For $(\nu, c) = (y, 1)$, we have $\tilde{\nu} = \big(y - b, \alpha\big) \in \mathscr{L}'$, with

$$\lambda_1(\mathscr{L}') \leqslant \|\tilde{\nu}\|_2 = \sqrt{\|y - b\|_2^2 + \alpha^2} \leqslant \sqrt{2}\alpha < \lambda_1(\mathscr{L}).$$

Now suppose that the vector

$$\nu' := \begin{pmatrix} \nu - cb \\ c\alpha \end{pmatrix} \in \mathscr{L}'$$

satisfies $\|\nu'\|_2 = \lambda_1(\mathscr{L}')$. We then consider three cases for $c \in \mathbf{Z}$:

† If $c = 0$, then $\|\nu'\|_2 = \|\nu\|_2 \geqslant \lambda_1(\mathscr{L}) > \lambda_1(\mathscr{L}')$, which is impossible.

† If $|c| \geqslant 2$, then $\|\nu'\|_2 \geqslant |c|\alpha \geqslant 2\alpha > \sqrt{2}\alpha \geqslant \lambda_1(\mathscr{L}')$, which is impossible.

† Therefore, $c = \pm 1$. If $c = -1$, replace ν' with $-\nu'$, reducing to the case $c = 1$. For $c = 1$, we have

$$\nu' = \begin{pmatrix} \nu - b \\ \alpha \end{pmatrix}, \quad \text{with } \nu \in \mathscr{L}.$$

If $\nu \neq y$, then

$$\|\nu - b\|_2 > \|y - b\|_2 \quad \Longrightarrow \quad \|\nu'\|_2 > \|\tilde{\nu}\|_2,$$

contradicting the fact that ν' is the shortest vector in $\mathscr{L}'$. Hence, necessarily, $\nu = y$.

We conclude that the only vectors of norm $\lambda_1(\mathscr{L}')$ in $\mathscr{L}'$ are $\pm\tilde{\nu}$, and that y can be recovered by projecting the first m-tuple of $\tilde{\nu}$ and adding b. □

4 Kyber and Dilithium

The post-quantum schemes Kyber (KEM) and Dilithium (signature) are mathematically based on the hardness of lattice problems, but with a particular structure: they use module lattices constructed over polynomial rings ($R_q = \mathbf{Z}_q[X]/(X^n + 1)$, where q is a prime number $\geqslant 3$ and $n \geqslant 1$ is an integer), which improves the efficiency of the method while preserving lattice-based security.

In Section 1, we introduced classical lattices: discrete subgroups of $\mathbf{R}^n$ generated by integer linear combinations of a basis, together with hard problems such as SVP, CVP, and GapSVP, which underpin security in post-quantum cryptography.

In Kyber and Dilithium, these constructions are transferred to the setting of module lattices, where vectors are replaced by polynomials in R_q and integer matrices by polynomial matrices. The underlying security problems are module-LWE and module-SIS variants, which generalise LWE and SIS while enabling fast operations, notably through ring arithmetic and the use of the NTT algorithm[1], for fast multiplication of polynomials in R_q.

The fundamental connection is as follows:

- The module-LWE problems used in Kyber rely on reductions from GapSVP over module lattices and provide security against both classical and quantum attacks.
- Similarly, Dilithium relies on module-SIS, the module analogue of SIS, and achieves strong resistance by relying on the difficulty of finding short vectors in module lattices.

[1] The time complexity of the NTT (Number-Theoretic Transform) algorithm is $O(n \log n)$ $\mathbf{Z}_q$-operations, compared with $O(n^2)$ $\mathbf{Z}_q$-operations using a classical algorithm.

4.1 Kyber

A *key-encapsulation mechanism* (Key Encapsulation Mechanism, KEM) is a cryptographic protocol that enables two parties to share a secret key securely, even over an insecure public communication channel. Its principle is as follows:

† The sender uses the recipient's public key to encapsulate (encrypt) a random session key.

† The recipient uses their private key to decapsulate (decrypt) and recover this session key.

† The two parties can then use this session key to encrypt their communications efficiently with fast symmetric encryption.

A KEM is more practical and secure than directly encrypting large messages with public-key schemes because:

† It encapsulates only a short key (e.g. 256 bits).

† This key can then be used with faster symmetric algorithms to encrypt the actual data.

† It ensures the confidentiality of the shared secret.

Kyber is a key-encapsulation mechanism resistant to quantum attacks. It was standardised by NIST in FIPS 203 under the name ML-KEM (Module-Lattice-based KEM). Kyber's security relies on the hardness of module-LWE problems over lattices, which are resistant to quantum attacks. In what follows, we use the ML-KEM-768 parameters:

† $q = 3329$: the modulus used for operations in the ring R_q.

† $n = 256$: the degree of the polynomial modulo $X^n + 1$ in the ring $R_q = \mathbf{Z}_q[X]/(X^n + 1)$, which defines the base dimension of the polynomial vectors used.

† $k = 3$: the number of columns of the polynomial matrices A, which controls the public-key size and the security–performance trade-off.

† $\eta_1 = 2$: the bound for sampling the secret vectors s_1 and s_2 with small norm in the signature algorithm.

† $\eta_2 = 2$: the bound used when sampling noise in the secret components of the encryption step (the encapsulated key).

† $d_u = 10$: the compression parameter for the ciphertext component u, used to reduce its size.

† $d_v = 4$: the compression parameter for the ciphertext component v, which also affects decapsulation verification.

Let $R_q := \mathbf{Z}_q[X]/(X^n + 1)$. For $\eta \in \mathbf{N}^*$, define

$$S_\eta := \{f \in R_q \mid \|f\|_\infty \leqslant \eta\}.$$

Definition 4.1 (Coefficient compression/decompression)

Let d be an integer such that $1 \leqslant d \leqslant \lfloor \log_2 q \rfloor$, with $q = 3329$ and $\log_2 q = 11$ because $q = 2^{11} + 2^{10} + 2^8 + 1$. Define the following two functions on $\{0, \cdots, q-1\}$ and $\{0, \cdots, 2^d - 1\}$:

$$\mathsf{Compress}_q(x, d) = \left\lceil \tfrac{2^d}{q} x \right\rfloor \bmod 2^d, \quad x \in \{0, \cdots, q-1\},$$

$$\mathsf{Decompress}_q(y, d) = \left\lceil \tfrac{q}{2^d} y \right\rfloor \bmod q, \quad y \in \{0, \cdots, 2^d - 1\}.$$

Remarks.

† Let $x \in \{0, 1, \cdots, q-1\}$. Set

$$x' := \mathsf{Decompress}_q\big(\mathsf{Compress}_q(x, d), d\big).$$

Then

$$\|x' - x\|_\infty \leqslant \lceil q/2^{d+1} \rceil.$$

† The functions $\mathsf{Compress}_q$ and $\mathsf{Decompress}_q$ extend naturally to the coefficients of polynomials in R_q and to polynomial vectors in R_q^k by applying them coordinate-wise.

Definition 4.2 (Centred binomial distribution (CBD_η))

Let η be a positive integer. The *centred binomial distribution* with parameter η is the distribution over $\{-\eta, \cdots, \eta\}$ defined as follows:

1. Independently sample η pairs of bits (a_i, b_i), for $i = 1, \cdots, \eta$, uniformly from $\{0,1\}^2$.
2. Set
$$c := \sum_{i=1}^{\eta} (a_i - b_i) \in \{-\eta, \cdots, \eta\}.$$
3. Then, for every $j \in \{-\eta, \cdots, \eta\}$,
$$\mathbf{P}(c = j) = \frac{\binom{2\eta}{\eta + j}}{2^{2\eta}}.$$

Remark. A polynomial can be sampled uniformly from S_η by sampling each of its coefficients uniformly from $\{-\eta, \cdots, \eta\}$. For simplicity, the coefficients c are sampled according to a *centred binomial distribution (CBD).*

We can now define Kyber's public-key encryption scheme (Kyber-PKE): the generation of public and private keys, the encryption of a message $m \in \{0,1\}^n$ of length n, and finally the decryption of the ciphertext to recover m. In what follows, we use the hash function[2]

$$\mathrm{SHAKE128} : \{0,1\}^{256} \longrightarrow \mathscr{M}_k(R_q),$$

to deterministically generate a pseudorandom matrix A from a seed ρ.

Kyber-PKE: generation of public and private keys

1. Select $\rho \longleftarrow \{0,1\}^{256}$ and compute:
$$A = \mathrm{SHAKE128}(\rho) \in \mathscr{M}_k(R_q).$$
2. Sample:
$$s \longleftarrow \mathscr{B}(S_{\eta_1}^k) \quad \text{and} \quad e \longleftarrow \mathscr{B}(S_{\eta_2}^k).$$
where $\mathscr{B}$ denotes the binomial distribution defined immediately above.
3. Compute
$$t = As + e.$$
4. Alice's public key (for encryption) is (ρ, t), and her private key (for decryption) is s.

Kyber-PKE: encryption of a message $m \in \{0,1\}^n$

1. Obtain an authentic copy of the public key (ρ, t) and compute
$$A = \mathrm{SHAKE128}(\rho) \in \mathscr{M}_k(R_q).$$
2. Sample
$$r \longleftarrow \mathscr{B}\big(S_{\eta_1}^k\big), \quad e_1 \longleftarrow \mathscr{B}\big(S_{\eta_2}^k\big), \quad e_2 \longleftarrow \mathscr{B}\big(S_{\eta_2}^k\big).$$
3. Compute
$$u = A^T r + e_1, \qquad v = t^T r + e_2 + \left\lceil \tfrac{q}{2} \right\rfloor m.$$
4. Compress
$$c_1 = \mathsf{Compress}_q(u, d_u), \qquad c_2 = \mathsf{Compress}_q(v, d_v).$$
5. Output the ciphertext $c = (c_1, c_2)$.

[2] See the standard [6] for further details on this hash function.

Kyber-PKE: decryption of the ciphertext $c = (c_1, c_2)$

1. Decompress
$$u' = \mathsf{Decompress}_q(c_1, d_u), \qquad v' = \mathsf{Decompress}_q(c_2, d_v).$$
2. Recover the message
$$m = \mathsf{Round}_q\big(v' - s^T u'\big).$$
where, for $x \in \{0, \cdots, q-1\}$, we define
$$\mathsf{Round}_q(x) := \begin{cases} 0 & \text{if } -q/4 < x \bmod q < q/4 \\ 1 & \text{otherwise} \end{cases} \quad \text{and} \quad x \bmod q := \begin{cases} x & \text{if } x \leqslant \frac{q-1}{2} \\ x - q & \text{if } x > \frac{q-1}{2} \end{cases}.$$

Remark. The size of the compressed ciphertext is therefore
$$3 \times 256 \times 10 + 256 \times 4 = 1088 \text{ bytes.}$$

Proposition 4.3 (Correctness of decryption)
Let $c = (c_1, c_2)$ be a ciphertext produced by the Kyber-PKE algorithm. Let
$$u' = u + e_u, \quad v' = v + e_v$$
denote the decompressed versions of c_1, c_2. Set
$$E = e^T r + e_2 - s^T e_1 + e_v - s^T e_u \in R_q$$
as the overall error polynomial. Then, if $\|E\|_\infty < \frac{q}{4}$, we indeed have
$$m = \mathsf{Round}_q\big(v' - s^T u'\big).$$

Proof. We start from
$$\begin{aligned} v' - s^T u' &= (v + e_v) - s^T(u + e_u) \\ &= \big(v - s^T u\big) + e_v - s^T e_u. \end{aligned}$$
By construction of the ciphertext,
$$v - s^T u = e^T r + e_2 - s^T e_1 + \lceil \tfrac{q}{2} \rfloor m.$$
Hence
$$v' - s^T u' = \lceil \tfrac{q}{2} \rfloor m + E.$$
If every coefficient E_i of E satisfies $-q/4 < E_i \bmod q < q/4$, then $\mathsf{Round}_q(\lceil q/2 \rfloor m + E) = m$ coordinate-wise, which proves the correctness of decryption. □

Remarks.

† For the ML-KEM-768 parameters, the coefficients E_i do not always satisfy $|E_i| < q/4$, and decryption is therefore not guaranteed to be failure-free. However, one can show that $\|E\|_\infty < q/4$ with probability very close to 1. Consequently, decryption succeeds with very high probability.

† Ciphertext compression does not affect the security of Kyber-PKE. Consequently, Kyber-PKE is indistinguishable under chosen-plaintext attack, assuming that the D-MLWE problem (the decision module-LWE problem) is hard.

Definition 4.4 (Key Encapsulation Mechanism (KEM))
A *KEM (Key Encapsulation Mechanism)* enables two parties to establish a shared secret key (see Figure 4). A KEM consists of three algorithms:

1. KeyGen: Alice runs this algorithm to generate an encapsulation key ek (public key) and a decapsulation key dk (private key).

2. Encaps: Bob uses Alice's encapsulation key ek to generate a secret key K and a ciphertext c, and then sends c to Alice.
3. Decaps: Alice uses her decapsulation key dk to recover K from the ciphertext c.

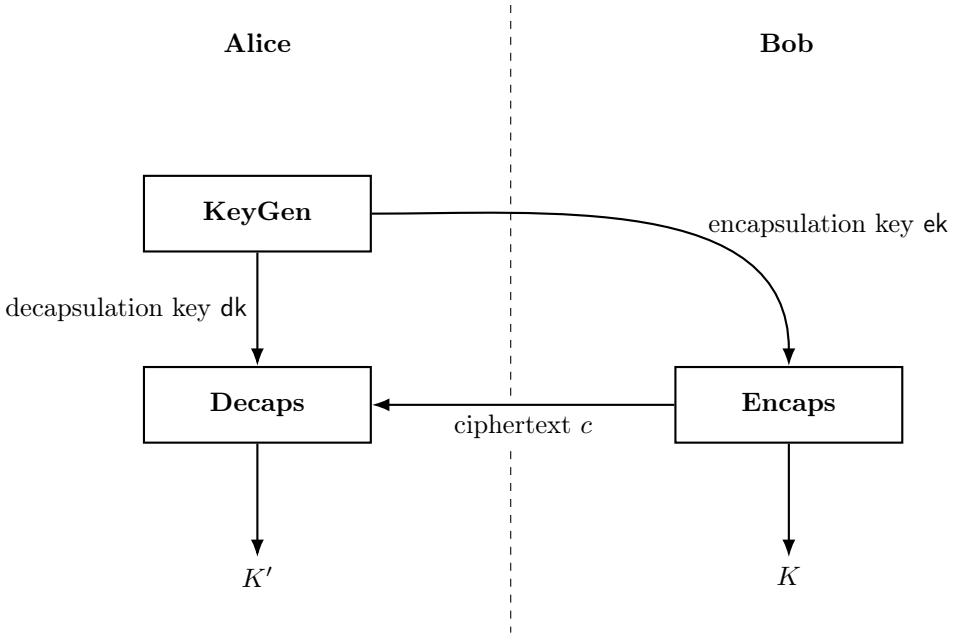


Figure 4: Key-Encapsulation Mechanism

We now define Kyber-KEM. This requires three hash functions[3]: SHA3-512 (denoted by G), SHA3-256 (denoted by H), and SHAKE256 (denoted by J).

Kyber-KEM: key generation

Alice proceeds as follows:

1. Use the Kyber-PKE key-generation algorithm to select a Kyber-PKE encryption key (ρ, t) and a decryption key s.
2. Uniformly select a secret z:
$$z \longleftarrow \mathscr{U}(\{0,1\}^{256}).$$
3. Alice's encapsulation key is $\mathsf{ek} = (\rho, t)$; her decapsulation key is:
$$\mathsf{dk} = (s, \mathsf{ek}, H(\mathsf{ek}), z).$$

Remark. The secret vector z is chosen once and for all and remains hidden; it is not used for encryption, but only during decapsulation to produce a fallback key without signalling an error.

Kyber-KEM: encapsulation

To establish a shared secret key with Alice, Bob proceeds as follows:

1. Obtain an authentic copy of Alice's encapsulation key ek.
2. Uniformly select a message m:
$$m \longleftarrow \mathscr{U}(\{0,1\}^{256}).$$

[3]See the standard [6] for further details on these hash functions.

3. Compute
$$h = H(\mathsf{ek}) \quad \text{and} \quad (K, R) = G(m, h),$$
where $K, R \in \{0,1\}^{256}$.

4. Use the Kyber-PKE encryption algorithm to encrypt m with the encryption key ek, using R to generate the required random quantities (via SHAKE256); denote the resulting ciphertext by c.

5. Output the secret key K and the ciphertext c.

Kyber-KEM: decapsulation

To recover the secret key K from c using $\mathsf{dk} = (s, \mathsf{ek}, H(\mathsf{ek}), z)$, Alice proceeds as follows:

1. Use the Kyber-PKE decryption algorithm to decrypt c with the decryption key s; denote the result by m'.

2. Compute
$$(K', R') = G(m', H(\mathsf{ek})).$$

3. Compute
$$\overline{K} = J(z, c).$$

4. Use the Kyber-PKE encryption algorithm to encrypt m' with the encryption key ek, using R' to generate the required random quantities (via SHAKE256); denote the resulting ciphertext by c'.

5. If $c \neq c'$, return $\overline{K}$.

6. Return K'.

Remarks.

† Decapsulation fails when $c \neq c'$. In this case, instead of returning an error, the secret vector z is used to compute the fallback key $\overline{K} = J(z, c)$, which is almost certainly different from the encapsulated key K, thereby ensuring CCA resistance without a rejection oracle.

† This may occur even when the communicating parties, Alice and Bob, behave honestly, because the underlying Kyber-PKE scheme has a very small failure probability (that is, when $m' \neq m$).

† The decapsulation failure rate can be shown to be negligible.

† Kyber-KEM is indistinguishable under chosen-ciphertext attack (IND-CCA), assuming that the D-MLWE problem is hard and that G, H, J are random functions.

† Kyber-KEM is also indistinguishable under a chosen-ciphertext attack conducted by a *quantum adversary* capable of making both classical and quantum (superposition) queries to the hash functions G, H, and J.

The following table presents the parameters and characteristics of the three ML-KEM security levels. For each level (512, 768, and 1024), it gives the values of the main algorithmic parameters $(q, n, k, \eta_1, \eta_2, d_u, d_v)$, together with the size of the encapsulation key (ek), the size of the ciphertext c in bytes, and the decapsulation failure rate, which remains negligible in every case.

	Security	q	n	k	η_1	η_2	d_u	d_v	size of ek (bytes)	size of c (bytes)	decapsulation failure rate
ML-KEM-512	1	3329	256	2	3	2	10	4	800	768	$< 2^{-139}$
ML-KEM-768	3	3329	256	3	2	2	10	4	1184	1088	$< 2^{-164}$
ML-KEM-1024	5	3329	256	4	2	2	11	5	1568	1568	$< 2^{-174}$

Remark. For Categories 1, 3, and 5, the fastest known attacks require at least as many resources as an exhaustive key search against 128-bit, 192-bit, and 256-bit block ciphers, respectively.

4.2 Dilithium

A *digital signature* is the electronic equivalent of a handwritten signature and makes it possible to:

† authenticate the sender of a message,

† ensure the integrity of the message,

† guarantee non-repudiation (the sender cannot deny having signed it).

The principle is as follows:

† The sender generates a signature on the message using their private key.

† Any recipient can verify the signature using the sender's public key, confirming that the message genuinely came from the sender and has not been altered.

Dilithium is a digital-signature scheme resistant to quantum computers. It was standardised by NIST in FIPS 204, where it is called ML-DSA (Module-Lattice-based Digital Signature Algorithm). In what follows, we consider:

† $\widetilde{S}_{\gamma_1}$, the set of polynomials in R_q with coefficients modulo q in $]-\gamma_1, \gamma_1]$,

† a pair of integers (k, ℓ) with $k > \ell$,

† B_τ, the set of polynomials in S_1 with exactly τ coefficients equal to ± 1,

† $\beta := \tau\eta$,

† γ_2 and $\alpha := 2\gamma_2$,

† λ, half the bit length of the signature component $\tilde{c}$,

† d, the number of bits removed from t (see Step 5 of key generation),

† ω, the maximum number of ones in h.

We choose the ML-DSA-87 parameters: $q = 2^{23} - 2^{13} + 1 = 8380417 \approx 2^{23}$, $n = 256$, $(k, \ell) = (8, 7)$, $\eta = 2$, $d = 13$, $\gamma_1 = 2^{19}$, $\tau = 60$, $\beta = 120$, $\gamma_2 = (q-1)/32 = 262144 \approx 2^{18}$, $\alpha = (q-1)/16 = 523776 \approx 2^{19}$, $\lambda = 256$, and $\omega = 75$.

The Dilithium signature scheme is as follows:

Key generation:

Alice proceeds as follows:

1. Select
$$\xi \longleftarrow \mathscr{U}(\{0,1\}^{256}).$$

2. Compute
$$(\rho, \rho', K) = H(\xi, 1024),$$
where $\rho \in \{0,1\}^{256}$, $\rho' \in \{0,1\}^{512}$, and $K \in \{0,1\}^{256}$.

3. Compute
$$A = \text{SHAKE128}(\rho) \in \mathscr{M}_{k,\ell}(R_q).$$

4. Compute
$$(s_1, s_2) = \text{SHAKE256}(\rho')$$
with $(s_1, s_2) \in S_\eta^\ell \times S_\eta^k$.

5. Compute
$$t = As_1 + s_2 \in R_q^k$$
with $t \in R_q^k$.

6. Compute

$$(t_1, t_0) = \text{Power2Round}(t, d),$$

where $\text{Power2Round}(t, d) := (r \bmod 2^d, \frac{r - r_0}{2^d})$.

7. Compute

$$\mathsf{tr} = (\rho \parallel t_1, 512),$$

where $\rho \parallel t_1$ denotes the concatenation of ρ and t_1.

8. Alice's verification key is

$$\mathsf{pk} = (\rho, t_1),$$

and her signing key is

$$\mathsf{sk} = (\rho, K, \mathsf{tr}, s_1, s_2, t_0).$$

Signature generation:

To sign a message $M \in \{0,1\}^*$, Alice proceeds as follows:

1. Compute

$$A = \text{SHAKE128}(\rho) \in \mathscr{M}_{k,\ell}(R_q).$$

2. Compute

$$\mu = H(tr \parallel M, 512).$$

3. Compute

$$\rho'' = H(K \parallel \text{rnd} \parallel \mu, 512),$$

where $\text{rnd} \longleftarrow \{0,1\}^{256}$.

4. $\kappa \longleftarrow 0$.

5. Found $\longleftarrow$ false.

6. While Found = false, do:

a) Compute

$$y = \text{SHAKE256}(\rho'', \kappa) \in \widetilde{S}_{\gamma_1}^{\ell}.$$

b) Compute

$$w = Ay, \quad w_1 = \text{HighBits}(w, \alpha),$$

where $\text{HighBits}(w_i, \alpha) = \frac{w_i - w_i \bmod \alpha}{\alpha}$ extracts the high-order bits of each coefficient of the polynomial $w \in R_q$, modulo q, using the rounding parameter α.

c) Compute

$$\tilde{c} = H(\mu \parallel w_1, 2\lambda), \quad c = \text{SampleInBall}(\tilde{c}) \in B_\tau,$$

where SampleInBall takes a bit string as input (here, $\tilde{c}$ obtained from a hash) and extracts a polynomial, or a vector of polynomials, c in B_τ.

d) Compute

$$z = y + cs_1.$$

e) Compute

$$r_0 = \text{LowBits}(w - cs_2, \alpha),$$

where

$$\text{LowBits}(w_i, \alpha) := w_i - \alpha \cdot \text{HighBits}(w_i, \alpha) \quad (= w_i \bmod \alpha)$$

extracts the low-order bits of $w - cs_2$ by subtracting the high-order bits.

f) If $\|z\|_\infty < \gamma_1 - \beta$ and $\|r_0\|_\infty < \gamma_2 - \beta$, then, if $\| - ct_0\|_\infty < \gamma_2$:

† Compute

$$h = \text{MakeHint}(-ct_0, w - cs_2 + ct_0, \alpha),$$

where we write

$$\text{MakeHint}(a, b, \alpha) := \begin{cases} 1 & \text{if HighBits}(a + b, \alpha) \neq \text{HighBits}(b, \alpha) \\ 0 & \text{otherwise} \end{cases},$$

which produces a binary indicator showing, for each coefficient, whether rounding b with threshold α differs from rounding a, so that the verifier can reconstruct the rounded value by applying UseHint without access to the low-order bits.

† If the number of ones in h is $\leqslant \omega$, then Found $\longleftarrow$ true.

g) $\kappa \longleftarrow \kappa + \ell$.

7. Return

$$\sigma = (\tilde{c}, z, h).$$

Signature verification:

To verify the signature $\sigma = (\tilde{c}, z, h)$ on the message M, Bob proceeds as follows:

1. Obtain an authentic copy of Alice's public key $\mathsf{pk} = (\rho, t_1)$.

2. Check that $\|z\|_\infty < \gamma_1 - \beta$ and that the number of ones in h is $\leqslant \omega$; otherwise, reject.

3. Compute

$$A = \text{SHAKE128}(\rho) \in \mathscr{M}_{k,\ell}(R_q).$$

4. Compute

$$\mathsf{tr} = H(\rho \parallel t_1, 512), \quad \mu = H(\mathsf{tr} \parallel M, 512).$$

5. Compute

$$c = \text{SampleInBall}(\tilde{c}).$$

6. Compute

$$w_1' = \text{UseHint}(h, Az - ct_1 2^d, \alpha),$$

where

$$\text{UseHint}(h, b, \alpha) := \text{HighBits}(b + z, \alpha).$$

7. Check that

$$\tilde{c} = H(\mu \parallel w_1', 2\lambda),$$

otherwise, reject.

8. Accept the signature.

Proposition 4.5 (Correctness (signature verification))
The signature scheme described above is correct.

Proof. We have

$$Az - ct_1 2^d = A(y + cs_1) - c(t - t_0) = Ay + cAs_1 - c(As_1 + s_2) + ct_0 = w - cs_2 + ct_0.$$

Since $\| - ct_0\|_\infty < \gamma_2$, we have

$$\begin{aligned} w_1' &= \text{UseHint}(h, Az - ct_1 2^d, \alpha) \\ &= \text{HighBits}(Az - ct_1 2^d - ct_0, \alpha) \\ &= \text{HighBits}(Az - ct, \alpha) \\ &= \text{HighBits}(w - cs_2, \alpha). \end{aligned}$$

Moreover, since

$$\|\text{LowBits}(w - cs_2, \alpha)\|_\infty < \gamma_2 - \beta \quad \text{and} \quad \|cs_2\|_\beta \leqslant \beta,$$

we obtain

$$\text{HighBits}(w - cs_2, \alpha) = \text{HighBits}(w, \alpha) = w_1.$$

Thus $w_1' = w_1$, which completes the proof. □

Remark. For Categories 2, 3, and 5, the fastest known attacks require at least as many resources as an exhaustive key search against, respectively, a 256-bit hash function, a 192-bit block cipher, and a 256-bit block cipher.

We now determine the number of iterations required for signing:

Let $p_1 = \mathbf{P}(\|z\|_\infty < \gamma_1 - \beta)$. We have $z = y + cs_1$, with $y \in \widetilde{S}_{\gamma_1}^{\ell}$. Suppose that a coefficient of cs_1 is $u \in \{-\beta, \cdots, \beta\}$. The corresponding coefficient of z lies in $]-\gamma_1 + \beta, \gamma_1 - \beta[$ if the coefficient of y lies in $(-\gamma_1 + \beta - u, \gamma_1 - \beta - u)$. This interval has length $2(\gamma_1 - \beta) - 1$, independently of u. Thus,

$$p_1 = \left(\frac{2(\gamma_1 - \beta) - 1}{2\gamma_1}\right)^{256\ell} = \left(1 - \frac{\beta + 1/2}{\gamma_1}\right)^{256\ell} \approx e^{-256\ell\beta/\gamma_1}.$$

Let $p_2 = \mathbf{P}(\|r_0\|_\infty < \gamma_2 - \beta)$. Assuming that $\text{LowBits}(w - cs_2, 2\gamma_2)$ is uniformly distributed:

$$p_2 = \left(\frac{2(\gamma_2 - \beta) - 1}{2\gamma_2 + 1}\right)^{256k} \approx \left(1 - \frac{\beta + 1/2}{\gamma_2}\right)^{256k} \approx e^{-256k\beta/\gamma_2}.$$

Let $p_3 = \mathbf{P}(\| - ct_0\|_\infty < \gamma_2$ and the number of ones in h is $\leqslant \omega)$. Then $p_3 > 0.98$.

The probability that the four conditions are satisfied is $p_1 p_2 p_3$.
Thus, the number of iterations during signing is:

$$\frac{1}{p_1 p_2 p_3} \approx \frac{1}{p_1 p_2} \approx e^{256\beta(\ell/\gamma_1 + k/\gamma_2)}.$$

The number of signing iterations is approximately 4.25 for ML-DSA-44, 5.1 for ML-DSA-65, and 3.85 for ML-DSA-87.

The following two tables present the parameters and object sizes for the different variants of the ML-DSA signature scheme.

	Security Category	q	n	(k, ℓ)	η	d	γ_1	τ	β	γ_2	λ	ω
ML-DSA-44	2	$2^{23} - 2^{13} + 1$	256	(4,4)	2	13	2^{17}	39	78	$\frac{q-1}{88}$	128	80
ML-DSA-65	3	$2^{23} - 2^{13} + 1$	256	(6,5)	4	13	2^{19}	49	196	$\frac{q-1}{32}$	192	55
ML-DSA-87	5	$2^{23} - 2^{13} + 1$	256	(8,7)	2	13	2^{19}	60	120	$\frac{q-1}{32}$	256	75

	Security Category	Signing key (bytes)	Verification key (bytes)	Signature (bytes)
ML-DSA-44	2	2560	1312	2420
ML-DSA-65	3	4032	1952	3309
ML-DSA-87	5	4896	2592	4627

4.3 Python Implementation of Dilithium

The following Python implementation captures the main ideas of Dilithium:

† generation of the public matrix,

† sampling of small secret vectors,

† computation of the public vector,

† signing and verification using a hashed challenge.

The selected security parameters are as follows:

† ML-DSA-44 : $k = 4$, $l = 4$, $\eta = 2$.

† ML-DSA-65 : $k = 6$, $l = 5$, $\eta = 2$.

† ML-DSA-87 : $k = 8$, $l = 7$, $\eta = 2$.

The hash functions are imported directly from the corresponding libraries. The code is available at the following GitHub link:

GitHub repository: Dilithium-Post-Quantum Cryptography

5 Appendix: Free Groups

Definition 5.1
An abelian group admitting a basis of n elements is called a free abelian group of rank n.

Remark. If $(G, +)$ is free abelian of rank n, and $(x_1, \cdots, x_n)$ and $(y_1, \cdots, y_n)$ are two bases of G, then there exist a_{ij}, $b_{ij} \in \mathbf{Z}$ such that

$$y_i = \sum_{j=1}^{n} a_{ij} x_j, \qquad x_i = \sum_{j=1}^{n} b_{ij} y_j.$$

If we consider the matrices $A = (a_{ij})_{1 \leqslant i,j \leqslant n}$ and $B = (b_{ij})_{1 \leqslant i,j \leqslant n}$, it follows that $AB = I_n$, the identity matrix. Hence

$$\det(A) \det(B) = 1$$

and, since $\det(A)$ and $\det(B)$ are integers, we have

$$\det(A) = \det(B) = \pm 1.$$

A square integer matrix with determinant ± 1 is said to be unimodular. We have the following lemma:

Lemma 5.2
Let G be a free abelian group of rank n with basis $(x_1, \cdots, x_n)$. Let $A = (a_{ij})_{1 \leqslant i,j \leqslant n} \in \mathscr{M}_n(\mathbf{Z})$. Then the elements

$$y_i = \sum_{j=1}^{n} a_{ij} x_j$$

form a basis of G if and only if the matrix A is unimodular.

Proof. We have already proved the forward implication.

Now suppose that A is unimodular. Since $\det(A) \neq 0$, the elements y_j are linearly independent over $\mathbf{Z}$. We have

$$A^{-1} = (\det(A))^{-1} \mathrm{Com}(A),$$

where $\mathrm{Com}(A)$ is the cofactor matrix, whose entries are integers. Thus $A^{-1} = \pm \mathrm{Com}(A)$ has integer entries. Setting $B = A^{-1} = (b_{ij})$, we obtain

$$x_i = \sum_{j=1}^{n} b_{ij} y_j,$$

which shows that the y_j generate G. Therefore, they do form a basis. □

The central result in the theory of finitely generated free abelian groups concerns the structure of their subgroups:

Theorem 5.3
Every subgroup H of a free abelian group G of rank n is free of rank $s \leqslant n$. Moreover, there exists a basis $(u_1, \cdots, u_n)$ of G and strictly positive integers $\alpha_1, \cdots, \alpha_s$ such that the family

$$(\alpha_1 u_1, \cdots, \alpha_s u_s)$$

is a basis of H.

Proof. We proceed by induction on the rank n of G. For $n = 1$, G is infinite cyclic, and the result follows from the structure of subgroups of a cyclic group. Suppose that G has rank n, and choose an arbitrary basis $(w_1, \cdots, w_n)$ of G. Then every $h \in H$ can be written as

$$h = h_1 w_1 + \cdots + h_n w_n, \quad \text{with } h_1, \cdots, h_n \in \mathbf{Z}.$$

Either $H = \{0\}$, in which case the theorem is trivial, or there exist coefficients $h_i \neq 0$ for some $h \in H$. Among all these non-zero coefficients, let $\lambda(w_1, \cdots, w_n)$ denote the smallest positive integer that occurs. Choose a basis $(w'_1, \cdots, w'_n)$ minimising $\lambda(w_1, \cdots, w_n)$, and denote this minimum by α_1. Renumber the w_i so that

$$v_1 = \alpha_1 w_1 + \beta_2 w_2 + \cdots + \beta_n w_n$$

belongs to H with coefficient α_1. For $2 \leqslant i \leqslant n$, write

$$\beta_i = \alpha_1 q_i + r_i, \quad 0 \leqslant r_i < \alpha_1,$$

so that r_i is the remainder when β_i is divided by α_1. Define

$$u_1 = w_1 + q_2 w_2 + \cdots + q_n w_n.$$

It is easy to verify that $(u_1, u_2, \cdots, u_n)$ is another basis of G (the change-of-basis matrix is unimodular). In this new basis, we have

$$v_1 = \alpha_1 u_1 + r_2 u_2 + \cdots + r_n u_n.$$

By the minimality of $\alpha_1 = \lambda(w_1, \cdots, w_n)$ over all bases, we obtain

$$r_2 = \cdots = r_n = 0,$$

hence

$$v_1 = \alpha_1 u_1.$$

With this basis, set

$$H' = \{m_1 u_1 + m_2 u_2 + \cdots + m_n u_n \mid m_1 = 0\}.$$

Clearly, $H' \cap V_1 = \{0\}$, where V_1 is the subgroup generated by v_1. Then $H = H' + V_1$. Indeed, for every $h \in H$, we have

$$h = \gamma_1 u_1 + \gamma_2 u_2 + \cdots + \gamma_n u_n,$$

and, setting

$$\gamma_1 = \alpha_1 q + r_1, \quad 0 \leqslant r_1 < \alpha_1,$$

it follows that H contains

$$h - q v_1 = r_1 u_1 + \gamma_2 u_2 + \cdots + \gamma_n u_n,$$

and the minimality of α_1 once again implies that $r_1 = 0$. Therefore $h - q v_1 \in H'$. It follows that H is isomorphic to $H' \times V_1$, and that H' is a subgroup of rank $n - 1$ of G', the free abelian group of rank $n - 1$ generated by $w_2, \cdots, w_n$.

By induction, H' has rank $\leqslant n - 1$, and there exist bases $u_2, \cdots, u_s$ of G' and $(v_2, \cdots, v_n)$ of H' such that $v_i = \alpha_i u_i$ for integers $\alpha_i > 0$.

The result follows. □

Theorem 5.4 (Structure of quotients of free abelian groups)

Let G be a free abelian group of rank r, and let H be a subgroup of G. Then the quotient G/H is finite if and only if H also has rank r.

In this case, if G and H admit respective $\mathbf{Z}$-bases $(x_1, \cdots, x_r)$ and $(y_1, \cdots, y_r)$, related by

$$y_i = \sum_{j=1}^{r} a_{i,j} x_j,$$

then

$$\big|G/H\big| = \big|\det\big(A\big)\big|,$$

where $A \in \mathscr{M}_r(\mathbf{Z})$ is the matrix with general entry $a_{i,j}$.

Proof. Let H have rank s, and use the preceding theorem to choose bases over $\mathbf{Z}$

$$(u_1, \cdots, u_r)$$

of G and

$$(v_1, \cdots, v_s)$$

of H such that

$$v_i = \alpha_i u_i \quad \text{for } 1 \leqslant i \leqslant s.$$

Clearly, G/H is the direct product of finite cyclic groups of orders $\alpha_1, \cdots, \alpha_s$ and $r - s$ infinite cyclic groups. Consequently, $|G/H|$ is finite if and only if $r = s$, and in this case

$$|G/H| = \alpha_1 \cdots \alpha_r.$$

We then have

$$u_i = \sum_{j=1}^{r} b_{ij} x_j, \qquad v_i = \sum_{j=1}^{r} c_{ij} u_j, \qquad y_i = \sum_{j=1}^{r} d_{ij} v_j,$$

where the matrices $(b_{ij}) = B$ and $(d_{ij}) = D$ are unimodular by Lemma 5.2, and

$$C = (c_{ij}) = \begin{pmatrix} \alpha_1 & 0 & \cdots & 0 \\ 0 & \alpha_2 & \ddots & \vdots \\ \vdots & \ddots & \ddots & 0 \\ 0 & \cdots & 0 & \alpha_r \end{pmatrix}.$$

Clearly, if $A = (a_{ij})$, then $A = BCD$, and therefore

$$\det(A) = \det(B) \det(C) \det(D).$$

Thus,

$$\big|\det(A)\big| = \big|\pm 1\big|\big|\det(C)\big|\big|\pm 1\big| = \big|\alpha_1 \cdots \alpha_r\big| = |G/H|,$$

as claimed. □